

ANÁLISIS Y DISEÑO DE UN MICRO-FRAMEWORK LC/NC PARA N8N CON
ARQUITECTURAS LIMPIAS Y DEVSECOPS:
EVALUACIÓN ATAM Y DISEÑO DE ARQUITECTURA EN AWS

ELIAN HERNANDO GIL SIERRA

UNIVERSIDAD AUTONOMA DE BUCARAMANGA
FACULTAD DE INGENIERIA
MAESTRIA EN GESTION, APLICACIÓN Y DESARROLLO DE SOFTWARE
BUCARAMANGA
2026

ELIAN HERNANDO GIL SIERRA

TRABAJO DE GRADO

DIRECTOR:
SEBASTIAN ROA PRADA, PhD

UNIVERSIDAD AUTONOMA DE BUCARAMANGA
FACULTAD DE INGENIERIA
MAESTRIA EN GESTION, APLICACIÓN Y DESARROLLO DE SOFTWARE
BUCARAMANGA
2026

CONTENIDO

	Pág.
1. INTRODUCCIÓN.....	12
2. ANTECEDENTES.....	13
3. OBJETIVOS	15
3.1 OBJETIVO GENERAL	15
3.2 OBJETIVOS ESPECIFICOS	15
4. MARCO TEÓRICO	16
4.1 PARADIGMA DE ABSTRACCIÓN Y DEUDA TÉCNICA EN LC/NC.....	16
4.2 CLEAN ARCHITECTURE Y MODULARIDAD EN FLUJOS DE DATOS.....	16
4.2.1 Estrategias de desacoplamiento y estratificación	18
4.2.2 Dinámicas de Interacción.....	18
4.2.3 Idempotencia y Resiliencia	18
4.2.4 Trazabilidad del Diseño mediante ADR (Architecture Decision Records).....	19
4.3 CICLO DE VIDA DEVSECOPS Y SEGURIDAD POR DISEÑO.....	19
4.4 EVALUACIÓN DE CALIDAD Y METODOLOGÍA ATAM.....	20
4.4.1 Definición de Escenarios de Evaluación (Top-K)	22
4.5 OPERACIÓN Y ESCALABILIDAD EN AWS.....	24
4.5.1 Pilares de la Implementación Cloud	24
5. METODOLOGIA PROPUESTA	26
5.1 ENFOQUE METODOLÓGICO	27
5.2 POBLACIÓN Y MUESTRA.....	27
5.3 TÉCNICAS E INSTRUMENTOS DE RECOLECCIÓN DE DATOS.....	28
5.3.1 Definiciones operativas	28
5.3.2 Componente cuantitativo: métricas y fuentes de datos	29
5.3.3 Protocolo de captura y trazabilidad de medición	31
5.4 PROCEDIMIENTOS.....	32
5.4.1. Planeación e instrumentación	32
5.4.2. Levantamiento y caracterización as-is	32
5.4.3. Diseño del micro-framework	33
5.4.4. Construcción del estado to-be	34

5.4.5. Evaluación arquitectónica por escenarios ATAM	35
5.4.6. Diseño de arquitectura en AWS alineada.....	36
6. RESULTADOS ESPERADOS	41
6.1 RESULTADOS APLICADOS.....	41
R1. Marco arquitectónico LC/NC para n8n (micro-framework) con reglas, metamodelo y guía de revisión.....	41
R2. Diseños comparativos de dos casos (as-is y to-be) con documentación completa de decisiones.....	41
R3. Diseño de arquitectura en AWS costo-eficiente y escalable alineado con el micro-framework.....	42
6.2 RESULTADOS METODOLÓGICOS Y DE VALIDACIÓN	43
R4. Protocolo e informe ATAM para evaluar arquitectura en LC/NC comparando as-is vs to-be.....	43
6.3 RESULTADOS DE TRANSFERENCIA Y ESTANDARIZACIÓN	43
R5. Guía práctica de buenas prácticas para n8n orientada a calidad, operación y adopción gradual del micro-framework.	43
6.4 RESULTADOS ACADÉMICOS	44
R6. Documento final del trabajo de grado.....	44
7. ACTIVIDADES.....	45
8. RECURSOS NECESARIOS	47
8.1 RECURSOS COMPUTACIONALES	47
8.2 SOFTWARE ESPECIALIZADO Y HERRAMIENTAS DE SOPORTE	47
8.3 INFRAESTRUCTURA Y SERVICIOS EN LA NUBE.....	48
8.4 RECURSOS DE SOPORTE Y CONECTIVIDAD.....	48
BIBLIOGRAFÍA.....	49
ANEXOS	54
ANEXO A: PRESUPUESTO	54
ANEXO B: CRONOGRAMA DE ACTIVIDADES.....	56
ANEXO C: FORMATOS DE REGISTRO.....	58
ANEXO D: DETALLE ACTIVIDADES.....	63

LISTA DE FIGURAS

Figura 1. Comparación entre Low-Code/No-Code y desarrollo tradicional. (Fuente: elaboración propia).	16
Figura 2. Diagrama de capas de Clean Architecture en vista de “cebolla”. (Fuente: CleanCoderBlog. Ejemplo de flujo de trabajo WhatsApp-n8n. Recuperado de: https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html Consultado el 11/02/2026).	17
Figura 3. Diagrama de DevSecOps (Fuente: Atlassian. DevSecOps diagram. Recuperado de: https://wac-cdn.atlassian.com/dam/jcr:5f26d67b-bed6-4be1-912b-4032de4d06b0/devsecops-diagram.png . Consultado el 11/01/2026.).	20
Figura 4. Ejemplo Árbol de utilidad ATAM de sistema. (Fuente: elaboración propia)	22
Figura 5. Diagrama de servicios de cómputo en AWS. (Fuente: AWS Whitepaper – Compute Services. Recuperado de: https://docs.aws.amazon.com/es_es/whitepapers/latest/aws-overview/compute-services.html . Consultado el 11/01/2026.)	24
Figura 6. Pilares de Implementación Cloud (Fuente: elaboración propia)	25
Figura 7. Metodología propuesta. (Fuente: elaboración propia).	26
Figura 8. Diagrama as-is conceptual (Fuente: elaboración propia)	33
Figura 9. Diagrama conceptual micro-framework (Fuente: elaboración propia).	34
Figura 10. Diagrama To-be conceptual (Fuente: elaboración propia).....	35
Figura 11. Resumen ATAM (Fuente: elaboración propia).	36
Figura 12. Diagrama de despliegue AWS de referencia. (Fuente: elaboración propia). ...	38
Figura 13. Comparativo as-is vs to-be (Fuente: elaboración propia).	39

LISTA DE TABLAS

Tabla 1. Matriz de atributos de calidad y Criterios de evaluacion	21
Tabla 2. Escenarios Top-K	23
Tabla 3. Matriz de métricas cuantitativas y fuentes de evidencia	30
Tabla 4. Operacionalización resumida por atributo.	32
Tabla 5. Estructura técnica del micro-framework	34
Tabla 6 Mapeo AWS, atributos, reglas, micro-frameworks y evidencias.	37
Tabla 7. Componentes micro-framework	41
Tabla 8. Indicadores de verificación de R1.	41
Tabla 9. Estructura de los paquetes	42
Tabla 10. Indicadores de verificación R2.	42
Tabla 11. Componentes del diseño	42
Tabla 12. Indicadores de verificación R3	42
Tabla 13. Artefactos generados	43
Tabla 14. Indicadores de verificación R4.	43
Tabla 15. Contenido principal de la guía práctica para n8n	44
Tabla 16. Resumen actividades principales	46
Tabla 17. Especificaciones mínimas computacionales	47
Tabla 18. Herramientas necesarias para el proyecto.	48

LISTA DE ANEXOS

Anexo	Título	Descripción
A	Presupuesto	Detalle del presupuesto estimado para la ejecución del proyecto, incluyendo partidas de gastos y recursos.
B	Cronograma de actividades	Diagrama de Gantt que detalla las actividades y tareas programadas a lo largo del proyecto, con fechas y plazos.
C	Formatos de Registro	Formatos propuestos para el registro de datos y actividades durante la investigación y desarrollo del proyecto.
D	Detalle Actividades	Descripción detallada de las actividades realizadas en cada fase del proyecto, incluyendo tareas y responsables.
E	Planteamiento del Problema	Documento que describe en detalle el planteamiento del problema de investigación que fundamenta el proyecto de grado.
F	Ficha Resumen Tesis	Ficha técnica con el resumen completo de la tesis, incluyendo objetivos, metodología y resultados clave.
G	Documento Aval Director	Carta o documento de aval firmado por el director del proyecto, confirmando la aprobación y supervisión del mismo.

Nota: Los documentos correspondientes a los **Anexos D, E, F y G** se encuentran en la carpeta comprimida "**Anexos**" adjunta a este documento de trabajo de grado. Para acceder a estos documentos, por favor descomprima el archivo y consulte los documentos enumerados dentro de la carpeta.

GLOSARIO

ADR (ARCHITECTURAL DECISION RECORD)

Documento breve y estructurado que registra una decisión arquitectónica relevante, su contexto, las alternativas consideradas y las consecuencias esperadas. Facilita la trazabilidad y la evolución de la arquitectura.

AD-HOC

Expresión utilizada para describir una solución, procedimiento o diseño elaborado para atender una necesidad puntual o un caso específico, generalmente construido sin un método estandarizado ni lineamientos formales previamente definidos. En el contexto de desarrollo de software, se asocia con implementaciones realizadas de manera no sistemática, con documentación limitada y baja consistencia, lo que puede dificultar su mantenibilidad, escalabilidad, trazabilidad y control.

ARQUITECTURA AS-IS / TO-BE

As-is describe el estado actual de una arquitectura existente; to-be representa la arquitectura objetivo-propuesta, diseñada para mejorar atributos de calidad específicos.

ARQUITECTURA LIMPIA (CLEAN ARCHITECTURE)

Enfoque arquitectónico que promueve la separación de responsabilidades mediante capas, independencia de frameworks y dependencias dirigidas hacia el núcleo del dominio, favoreciendo mantenibilidad y testabilidad.

ATAM (ARCHITECTURE TRADEOFF ANALYSIS METHOD)

Método sistemático de evaluación arquitectónica basado en escenarios, utilizado para analizar cómo las decisiones de diseño impactan atributos de calidad y para identificar riesgos y compromisos (trade-offs).

ATRIBUTOS DE CALIDAD

Características no funcionales de un sistema, como mantenibilidad, seguridad, confiabilidad, escalabilidad, trazabilidad y gobernanza, que determinan su comportamiento y sostenibilidad en el tiempo.

CLOUD / COMPUTACIÓN EN LA NUBE

Modelo de provisión de recursos tecnológicos (cómputo, almacenamiento, redes, servicios) bajo demanda, escalables y medidos por uso, generalmente ofrecidos por proveedores como AWS.

DEVSECOPS

Evolución de DevOps que integra la seguridad como un componente transversal y continuo del ciclo de vida del software, incorporando controles, validaciones y gestión de riesgos desde el diseño.

GOBERNANZA ARQUITECTÓNICA

Conjunto de reglas, procesos y mecanismos que aseguran que las decisiones de diseño, implementación y operación estén alineadas con los objetivos del sistema y se mantengan coherentes en el tiempo.

IDEMPOTENCIA

Propiedad de una operación que permite ejecutarla múltiples veces sin generar efectos secundarios adicionales, clave para la confiabilidad de flujos automáticos y reintentos.

LOW-CODE / NO-CODE (LC/NC)

Paradigma de desarrollo que permite construir soluciones de software mediante configuraciones visuales y mínima o nula programación tradicional, priorizando velocidad y accesibilidad.

MICRO-FRAMEWORK ARQUITECTÓNICO

Conjunto ligero de principios, reglas y patrones que orientan el diseño y organización de soluciones, sin imponer una arquitectura rígida ni herramientas específicas.

N8N

Plataforma open source de automatización Low-Code que permite orquestar flujos de trabajo mediante nodos visuales, integrando múltiples servicios y sistemas.

OBSERVABILIDAD

Capacidad de comprender el estado interno de un sistema a partir de sus salidas externas, apoyada en métricas, logs y trazas, para facilitar monitoreo, diagnóstico y operación.

PIPELINE DE DATOS

Flujo estructurado de procesamiento de datos que incluye etapas como ingesta, validación, transformación, persistencia y notificación.

TRAZABILIDAD

Capacidad de relacionar requerimientos, decisiones arquitectónicas, elementos de diseño y atributos de calidad, permitiendo justificar y auditar la arquitectura.

TRADE-OFF

Compromiso entre atributos de calidad, donde la mejora de uno puede implicar el deterioro de otro, y que debe ser analizado y documentado explícitamente.

UTILITY TREE

Artefacto de ATAM que organiza atributos de calidad en escenarios priorizados, facilitando la evaluación sistemática de una arquitectura.

RESUMEN

El presente proyecto tiene como propósito analizar y diseñar un micro-framework orientado a soluciones Low-Code/No-Code (LC/NC) en la plataforma n8n, fundamentado en principios de Clean Architecture y prácticas DevSecOps. El proyecto surge a partir de una tensión recurrente en este tipo de entornos: la alta velocidad de desarrollo y la baja barrera de entrada frente a las dificultades para garantizar atributos de calidad como mantenibilidad, trazabilidad, seguridad, confiabilidad y escalabilidad cuando los flujos se construyen de manera ad-hoc y sin lineamientos arquitectónicos claros.

En respuesta a esta problemática, se propone un marco arquitectónico que traduzca conceptos tradicionales de arquitectura de software al contexto visual y operativo de n8n, conectando decisiones de diseño con atributos de calidad mediante reglas, patrones y criterios aplicables a flujos LC/NC. La propuesta se valida a través de una evaluación arquitectónica basada en ATAM, comparando diseños as-is y to-be en dos casos de estudio representativos, con el fin de identificar impactos, riesgos y trade-offs entre rapidez de entrega y calidad del software.

De manera complementaria, el anteproyecto plantea un diseño de arquitectura en AWS alineado con el micro-framework, enfocado en la eficiencia de costos, la escalabilidad y la gobernanza operativa, incorporando consideraciones de seguridad, observabilidad y operación en nube. Como resultado esperado, se busca aportar un enfoque sistemático y replicable que permita evaluar hasta qué punto las soluciones LC/NC pueden aproximarse a estándares de calidad del desarrollo tradicional, manteniendo su ventaja competitiva en términos de agilidad.

PALABRAS CLAVE: Low-Code/No-Code, n8n, Arquitectura de Software, Clean Architecture, DevSecOps, Evaluación Arquitectónica, ATAM, Atributos de Calidad; Arquitectura en la Nube, AWS.

1. INTRODUCCIÓN

La adopción masiva de plataformas Low-Code/No-Code (LC/NC) ha cambiado la forma de construir software, principalmente al reducir los tiempos de entrega y eliminar las barreras técnicas de entrada. Herramientas como n8n permiten hoy que las empresas automaticen procesos complejos de manera visual y ágil. Sin embargo, esta rapidez suele traer un problema recurrente: muchos de estos flujos se diseñan de forma improvisada, sin seguir lineamientos claros de arquitectura o seguridad. Al final, esto termina comprometiendo la calidad del software, dificultando temas clave como el mantenimiento, la escalabilidad y la confiabilidad de la solución a largo plazo.

En el día a día, esta situación crea un conflicto evidente entre la velocidad y la calidad. Mientras que el enfoque LC/NC busca resultados inmediatos, la falta de reglas arquitectónicas y de prácticas operativas sistemáticas aumenta el riesgo de generar deuda técnica y costos operativos imprevistos. Bajo este panorama, surge una pregunta fundamental: ¿hasta qué punto se pueden estandarizar buenas prácticas en soluciones LC/NC para alcanzar niveles de calidad profesional sin perder la agilidad que estas plataformas ofrecen?

Con esto en mente, este proyecto propone el diseño de un micro-framework para n8n que adapte principios de Clean Architecture y prácticas DevSecOps al entorno del bajo código. Para validar la utilidad de esta propuesta, se utiliza el método de evaluación arquitectónica ATAM, comparando los diseños actuales (as-is) frente a los optimizados (to-be) en dos casos de estudio. El objetivo es identificar con claridad los impactos, riesgos y equilibrios que surgen al tomar estas decisiones de diseño.

Como complemento, se plantea una arquitectura en la nube sobre AWS alineada con este micro-framework. Esto permitirá observar el impacto real en términos de costos, seguridad y operación. En definitiva, el proyecto busca entregar un enfoque sistemático que sirva como guía para diseñar, evaluar y escalar soluciones LC/NC en entornos productivos reales.

2. ANTECEDENTES

En los últimos años, las plataformas Low-Code/No-Code (LC/NC) se han vuelto populares porque solucionan algo muy común en las empresas, la necesidad de crear soluciones rápido y automatizar tareas que cambian todo el tiempo. Esto no es solo una moda. Gartner proyectó que para 2025 el 70% de las nuevas aplicaciones desarrolladas por organizaciones incorporaría tecnologías LC/NC (Gartner, Inc., 2021). También estimó para 2023 un mercado de USD 26,9 mil millones en tecnologías low-code, con un crecimiento cercano al 20% frente a 2022 (Gartner, Inc., 2022). En la misma línea, se reporta que el 87% de los desarrolladores empresariales usa low-code en alguna parte de su trabajo (Bratincevic et al., 2024). Y en muchas empresas esto se adopta por partes, cada área elige lo que le sirve, así que terminan usando varias plataformas al tiempo, y eso complica el control y el orden (Mendix, 2025).

Dentro de ese panorama, la automatización e integración es de lo más usado porque permite conectar sistemas, mover datos y hacer que tareas se ejecuten solas sin tener que programar todo desde cero. Herramientas como n8n encajan ahí porque dan la posibilidad de armar flujos de trabajo de forma visual y cuando se necesita, también permiten agregar lógica más técnica. Esto ha impulsado su adopción y visibilidad, reflejada en señales de mercado y en su uso en despliegues reales (Highland Europe, 2025; n8n, 2025; Docker, Inc., n.d.).

El problema es que ir rápido no siempre significa hacerlo bien. Cuando crear flujos se vuelve tan fácil, también es fácil que se construyan incorrectamente y sin reglas claras. Con el tiempo empiezan a aparecer dolores típicos, como flujos difíciles de entender, cambios que rompen otras partes, validaciones hechas a medias, errores mal manejados, y poca trazabilidad de por qué se diseñó algo de cierta forma. Además, si muchas personas o equipos diferentes construyen automatizaciones, los riesgos crecen. La literatura sobre citizen development advierte que, sin gobernanza, pueden aparecer shadow IT, deuda técnica y problemas de calidad, sobre todo cuando no hay estándares ni supervisión (Binzer et al., 2024; Viljoen et al., 2024). Desde la industria también se insiste en que hacen falta prácticas de seguridad y gestión para que estas soluciones no se vuelvan un riesgo operativo (Microsoft, 2025).

Desde lo académico, también se reconoce esa tensión entre velocidad y control. Revisiones sistemáticas muestran que LC/NC trae beneficios claros, pero aún hay vacíos cuando se intenta evaluarlas con criterios comparables al desarrollo tradicional, especialmente en mantenibilidad, seguridad, confiabilidad y trazabilidad (Ajimati et al., 2025). En otras palabras, muchas soluciones funcionan, pero no siempre están listas para crecer y sostenerse en el tiempo sin convertirse en un problema (Trieflinger et al., 2025).

Con base en esto, el proyecto plantea diseñar un micro-framework para n8n que ayude a construir flujos más ordenados y repetibles, aplicando principios de arquitectura y prácticas DevSecOps dentro de un entorno LC/NC. La evaluación se apoya en ATAM, un método para analizar arquitecturas con escenarios y evidenciar riesgos y trade-offs en atributos de calidad (Kazman et al., 2000). La necesidad de este enfoque también se sostiene en que los riesgos de seguridad son reales, incluyendo vulnerabilidades reportadas en componentes relacionados con automatización y ejecución de código (GitHub, 2025; National Institute of Standards and Technology, 2025). Finalmente, se propone un diseño de referencia en AWS para aterrizar la solución en un entorno operativo realista, considerando seguridad, observabilidad y costos (Amazon Web Services, Inc., 2024).

3. OBJETIVOS

3.1 OBJETIVO GENERAL

Investigar y desarrollar un micro-framework para soluciones Low-Code/No-Code en n8n, fundamentado en principios de Clean Architecture y prácticas DevSecOps, con el propósito de estructurar y evaluar arquitecturas de software mediante la comparación de diseños as-is y to-be usando ATAM, analizar su impacto en atributos de calidad como mantenibilidad, trazabilidad, seguridad y confiabilidad, y orientar el diseño de arquitecturas costo-eficientes y escalables en AWS.

3.2 OBJETIVOS ESPECIFICOS

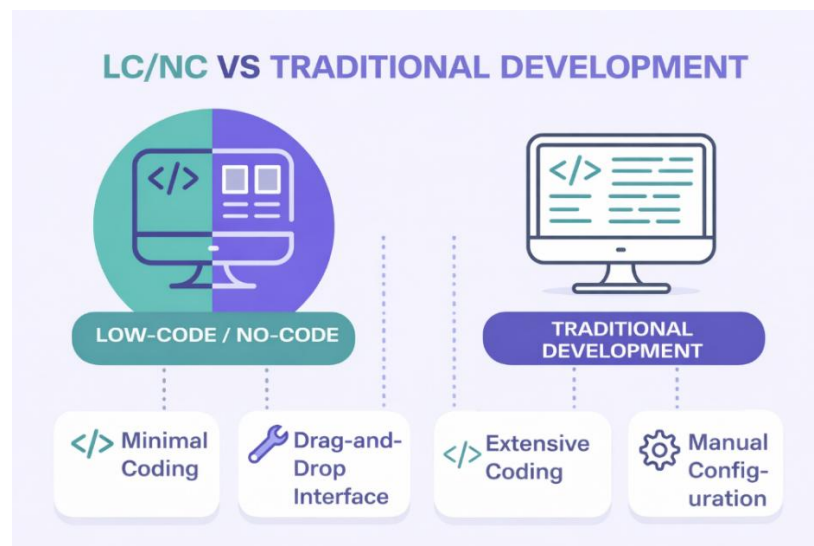
- Proponer y conceptualizar un marco arquitectónico para flujos Low-Code/No-Code en n8n, basado en principios de arquitectura limpia, que relacione atributos de calidad con decisiones de diseño, reglas de organización, patrones y criterios de buenas prácticas.
- Evaluar el uso del marco arquitectónico en dos casos de estudio representativos, documentando arquitecturas as-is y to-be con diagramas claros, registro de decisiones (ADR) y una matriz de trazabilidad entre requerimientos, decisiones y atributos de calidad.
- Aplicar un protocolo de evaluación arquitectónica basado en ATAM que permita analizar y comparar impactos, riesgos y compromisos de diseño entre las arquitecturas as-is y to-be de los casos de estudio seleccionados.
- Diseñar una arquitectura en AWS, costo-eficiente y escalable, alineado con el microframework, que describa opciones de despliegue, puntos de escalado, controles de seguridad, observabilidad y operación para soportar adopción y evolución en entornos productivos.

4. MARCO TEÓRICO

El sustento conceptual de este trabajo establece los pilares técnicos y de arquitectura necesarios para desarrollar un micro-framework sobre n8n. No se trata solo de una guía de uso, sino de una transición estructurada desde el desarrollo visual hacia una ingeniería de software rigurosa, apoyada en estándares de calidad, seguridad y observabilidad.

4.1 PARADIGMA DE ABSTRACCIÓN Y DEUDA TÉCNICA EN LC/NC

El auge de las plataformas Low-Code/No-Code (LC/NC) ha supuesto un cambio de paradigma orientado a minimizar la fricción técnica en las organizaciones (Hoogsteen & Borgman, 2022). Sin embargo, la agilidad que caracteriza a este modelo conlleva un riesgo crítico: la generación de una deuda técnica masiva cuando no se definen límites de diseño claros (Ajimati et al., 2025; Trieflinger et al., 2022).



*Figura 1. Comparación entre Low-Code/No-Code y desarrollo tradicional.
(Fuente: elaboración propia).*

La literatura es enfática al advertir que, sin una gobernanza adecuada, la velocidad inicial de despliegue se traduce pronto en flujos difíciles de mantener e integraciones inestables (Binzer et al., 2024; Viljoen et al., 2024). Por ello, este proyecto se aleja de la creación de flujos ad-hoc para enfocarse en reglas de juego que aseguren la permanencia de las soluciones bajo criterios de mantenibilidad y trazabilidad (Trieflinger et al., 2025).

4.2 CLEAN ARCHITECTURE Y MODULARIDAD EN FLUJOS DE DATOS

La aplicación de Clean Architecture (Martin, 2012) en entornos de bajo código surge como una respuesta técnica a la opacidad que suelen presentar los diseños visuales. El objetivo

principal es la separación de responsabilidades, logrando que las reglas de negocio, lo que realmente genera valor no dependan de los caprichos de la tecnología o de los conectores externos (Microsoft, n.d.).

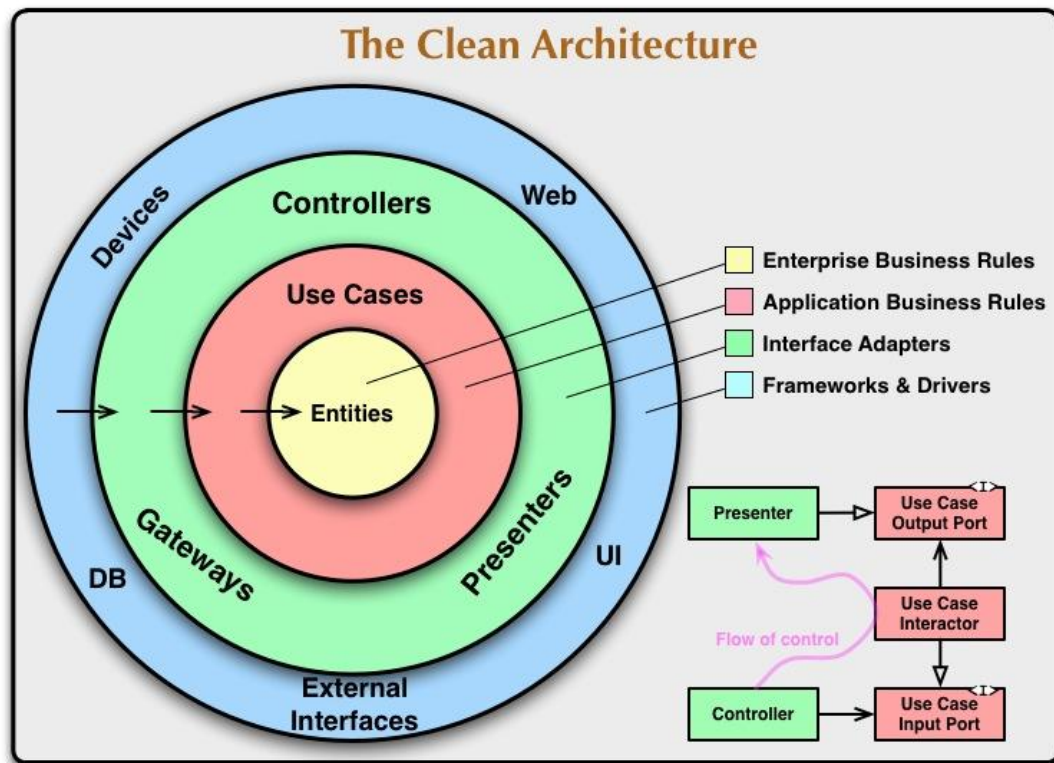


Figura 2. Diagrama de capas de Clean Architecture en vista de "cebolla".
(Fuente: CleanCoderBlog. Ejemplo de flujo de trabajo WhatsApp-n8n. Recuperado de: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html> Consultado el 11/02/2026).

En este micro-framework para n8n, aplicar Clean Architecture implica traducir esos principios al diseño de flujos, separar validaciones, reglas de negocio, integración y salidas, estandarizar entradas y salidas, al mismo tiempo definir convenciones sobre dónde viven decisiones críticas. Aunque el flujo sea visual, sigue siendo software, tiene dependencias, estados, errores y efectos colaterales. Por eso, si se busca calidad, se necesitan límites explícitos.

Este enfoque se complementa con principios de microservicios, la idea no es convertir n8n en microservicios, sino adoptar modularidad, alta cohesión, bajo acoplamiento, límites claros y responsabilidad única (Zhu & Cai, 2024). En la práctica, esto se puede reflejar en una orquestación de micro-flujos donde cada uno actúa como un servicio con tarea definida, validar, transformar, consultar o notificar. Con esto se evita que un solo flujo se convierta en un bloque gigante difícil de probar y mantener.

4.2.1 Estrategias de desacoplamiento y estratificación

Para este micro-framework, se han definido los siguientes principios de diseño:

- Independencia tecnológica: La lógica central debe permanecer aislada y protegida ante posibles cambios en los nodos de entrada o salida (Martin, 2012).
- Modularidad y micro-flujos: Se apuesta por una alta cohesión y bajo acoplamiento, fragmentando procesos complejos en una orquestación de micro-servicios con responsabilidades únicas (Zhu & Cai, 2024).
- Arquitectura Hexagonal: Mediante el uso de nodos personalizados, se encapsulan los detalles de integración para que los flujos dependan de abstracciones lógicas y no de implementaciones técnicas específicas.

4.2.2 Dinámicas de Interacción

En el diseño de sistemas distribuidos y flujos de automatización, la forma en que los componentes se comunican suele responder a dos patrones fundamentales: la coreografía, donde la autonomía de cada nodo dicta la acción basada en eventos, o la orquestación, donde un controlador central asume el mando del flujo (Zhu & Cai, 2024).

Para este micro-framework, se ha optado por un esquema de orquestación centralizada a través del nodo Execute Workflow. Esta elección no es arbitraria; responde a la necesidad de garantizar una visibilidad integral del proceso. Al centralizar el mando, el estado completo de la operación se vuelve rastreable en un único punto, lo que simplifica drásticamente la observabilidad mediante AWS CloudWatch. Asimismo, este enfoque facilita una gestión de errores mucho más robusta, permitiendo desplegar lógicas de fallback y reintentos uniformes que evitan que un fallo pase desapercibido en la complejidad de flujos desacoplados.

4.2.3 Idempotencia y Resiliencia

Un pilar fundamental para cualquier automatización es la idempotencia. Conceptualmente, nos referimos a la capacidad de una operación para ser ejecutada repetidamente sin alterar el resultado inicial.

Dentro del ecosistema de n8n, el micro-framework exige diseños idempotentes como salvaguarda ante ejecuciones duplicadas, un problema común frente a inestabilidades de

red o reintentos automáticos. En un entorno crítico como el propuesto, la resiliencia debe ir más allá de la idempotencia e integrar una estrategia clara de recuperación ante desastres. Esto exige implementar replicación de datos y redundancia en nodos clave para eliminar puntos únicos de falla (Single Point of Failure). Ante incidentes masivos, el sistema debe permitir una recuperación incremental que asegure el retorno al último estado coherente de forma controlada y segura.

Para lograrlo, se implementan validaciones de estados previos y claves de unicidad en las capas de persistencia. Esto asegura que, ante un evento inesperado como un timeout de API, el reintento del sistema no derive en inconsistencias de datos ni en la duplicidad de acciones críticas, como podría ser el envío múltiple de una misma notificación al usuario.

4.2.4 Trazabilidad del Diseño mediante ADR (Architecture Decision Records)

Para evitar que el diseño original se desvanezca con el tiempo fenómeno conocido como erosión arquitectónica y asegurar la transparencia en cada paso, se ha integrado el estándar de ADR, propuesto inicialmente por Michael Nygard. Un registro de decisión arquitectónica no es solo un documento; es una captura del contexto técnico, la decisión tomada y los compromisos o trade-offs resultantes.

La adopción de ADRs eleva el micro-framework de una simple estructura estática a un repositorio vivo de conocimiento. Esto permite que cualquier actor futuro comprenda las motivaciones técnicas detrás de cada restricción impuesta por el framework, convirtiendo la documentación en una herramienta estratégica de gobernanza.

4.3 CICLO DE VIDA DEVSECOPS Y SEGURIDAD POR DISEÑO

En un entorno donde las automatizaciones gestionan constantemente credenciales y datos sensibles, la seguridad no puede ser un elemento secundario. Es necesario integrar prácticas DevSecOps que eliminen los silos operativos y conviertan la protección de datos en un proceso continuo (Feio et al., 2024).

DevSecOps

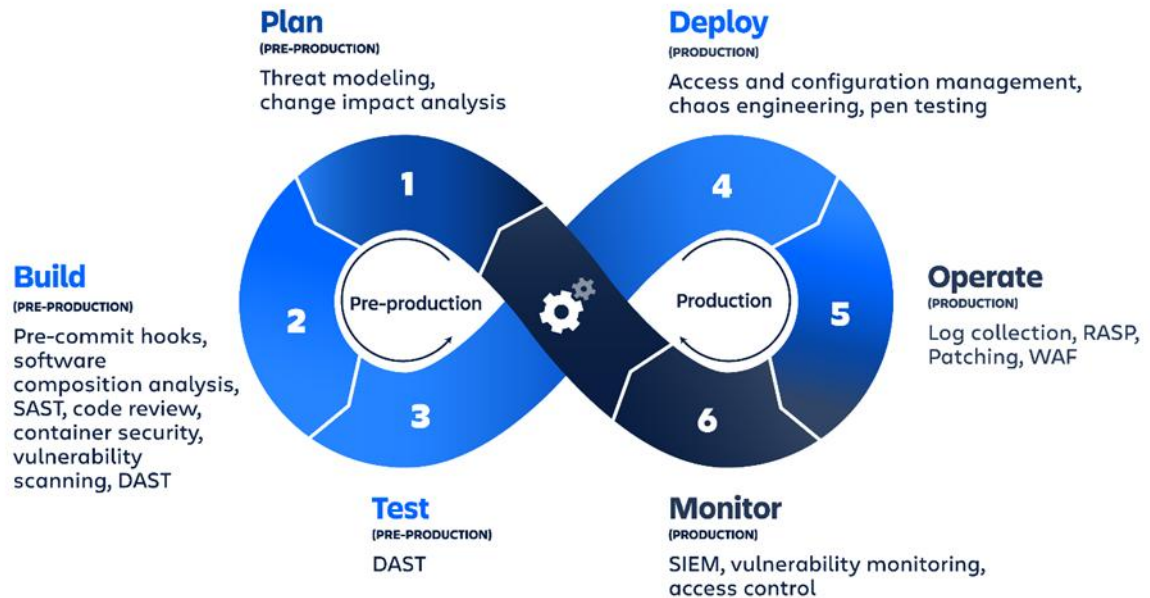


Figura 3. Diagrama de DevSecOps (Fuente: Atlassian. DevSecOps diagram. Recuperado de: <https://wac-cdn.atlassian.com/dam/jcr:5f26d67b-bed6-4be1-912b-4032de4d06b0/devsecops-diagram.png>. Consultado el 11/01/2026.)

Bajo esta premisa, el micro-framework operacionaliza tres pilares:

- Gestión de Secretos: Uso de bóvedas de credenciales para evitar que llaves API o datos críticos queden expuestos en los nodos.
- Validaciones Automatizadas: Implementación de análisis estáticos sobre la estructura de los flujos dentro del pipeline de despliegue.
- Resiliencia Operativa: Diseño de manejos de errores robustos que prevengan fallos en cascada en procesos críticos.

4.4 EVALUACIÓN DE CALIDAD Y METODOLOGÍA ATAM

Para sustentar que un micro-framework mejora diseño y operación, necesita un marco de evaluación que vaya más allá de opiniones. ISO/IEC 25010 aporta un modelo de calidad con características y subcaracterísticas que permiten discutir atributos de forma más objetiva (ISO/IEC, 2011). En este contexto, no se trata de evaluar todo, sino de priorizar atributos que suelen tensionarse en LC/NC:, como lo son la mantenibilidad, confiabilidad, seguridad, eficiencia y compatibilidad, entre otros.

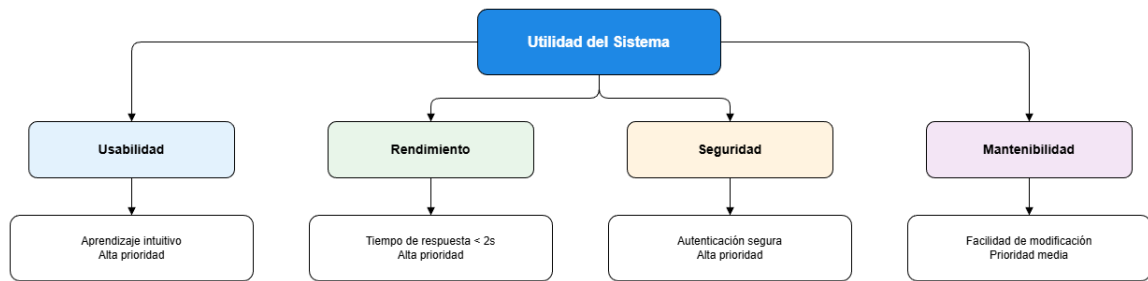
Característica (ISO 25010)	Sub-característica	Criterio Técnico en el Micro-framework	Métrica / Evidencia de Evaluación
Mantenibilidad	Modularidad	Grado de desacoplamiento entre flujos de negocio y nodos de integración (adaptadores).	Reducción en el número de nodos afectados ante un cambio en una API externa.
Mantenibilidad	Reusabilidad	Capacidad de los micro-flujos (vía <i>Execute Workflow</i>) para ser invocados por múltiples orquestadores.	Ratio de flujos compartidos vs. flujos totales.
Fiabilidad	Tolerancia a fallos	Implementación de patrones de reintento (<i>Retry</i>) y colas de errores (<i>Dead Letter</i>) centralizadas.	Porcentaje de recuperaciones exitosas ante fallos transitorios de red.
Seguridad	Confidencialidad	Aislamiento de secretos y uso de mecanismos de cifrado nativos/hube para credenciales.	Ausencia de datos sensibles (llaves API, tokens) en el código plano del JSON del flujo.
Eficiencia de Desempeño	Comportamiento temporal	Impacto de la modularización y el paso de datos entre flujos en el tiempo total de ejecución.	Latencia adicional introducida por el micro-framework vs. flujo monolítico tradicional.
Operabilidad	Capacidad de ser monitoreado	Integración de logs estructurados que permitan la trazabilidad total del proceso en CloudWatch.	Tiempo promedio de detección de fallos (MTTD) mediante alertas automáticas.

Tabla 1. Matriz de atributos de calidad y Criterios de evaluación

Complementariamente, el método ATAM (Architecture Tradeoff Analysis Method) permite evaluar la arquitectura mediante escenarios reales de uso (Kazman et al., 2000). Esta técnica es vital para:

- Detectar puntos sensibles: Identificar qué decisiones de diseño impactan directamente la calidad del sistema.

- Analizar Trade-offs: Evaluar, por ejemplo, si el aumento en la latencia debido a la modularización se compensa con una mayor facilidad de mantenimiento.
- Árboles de Utilidad: Priorizar las necesidades reales del sistema frente a los riesgos detectados (Kazman et al., 2000).



*Figura 4. Ejemplo Árbol de utilidad ATAM de sistema.
(Fuente: elaboración propia)*

Como resultado, ATAM produce un conjunto de riesgos arquitectónicos, puntos sensibles y trade-offs explícitos. Para este trabajo, esos resultados sirven como insumo directo para justificar el micro-framework, no solo se ve más ordenado, sino que reduce ciertos riesgos o mejora ciertos escenarios, y deja claros los costos o complejidades introducidas. Con esto, ISO/IEC 25010 da el lenguaje de calidad y ATAM da el método para relacionar calidad con decisiones.

4.4.1 Definición de Escenarios de Evaluación (Top-K)

Los siguientes escenarios representan las situaciones de mayor riesgo e impacto para los casos de estudio (Bot y IoT). Se han estructurado bajo el estándar de Kazman et al. (2000):

ID	Atributo de Calidad	de Escenario de Estímulo	Respuesta del Sistema (To-Be)	Medida de Respuesta
ESC-01	Mantenibilidad (Evolución)	Un cambio en la lógica de negocio (p.ej. cambiar el umbral de alerta en el pipeline IoT).	El cambio se realiza exclusivamente en la Capa de Dominio , sin alterar los nodos de integración.	Impacto de cambio: < 10% de los nodos totales modificados.
ESC-02	Mantenibilidad (Acoplamiento)	Un proveedor externo (p.ej.	Solo se modifica el Adaptador	Tiempo de cambio:

		API de WhatsApp) cambia su esquema de datos o versión de endpoint.	correspondiente; la lógica de orquestación y negocio permanece intacta.	Reducción del 30% vs As-Is.
ESC-03	Fiabilidad (Resiliencia)	Se presenta un <i>timeout</i> intermitente en el servicio de persistencia de datos durante una ingesta masiva.	El orquestador ejecuta la política de reintentos e idempotencia , evitando duplicidad de registros.	Tasa de fallos: 0 registros duplicados y 100% de trazabilidad del error.
ESC-04	Seguridad (Confidencialidad)	Se realiza una auditoría sobre el archivo JSON exportado del flujo de trabajo.	El archivo no contiene secretos, llaves API ni tokens embebidos (hardcoded).	Exposición: 0 hallazgos de datos sensibles.
ESC-05	Desempeño (Sobrecarga)	El sistema procesa ráfagas de datos simultáneos (p.ej. múltiples sensores reportando al tiempo).	El flujo mantiene la latencia P95 dentro de márgenes aceptables a pesar de la segmentación en sub-flujos.	Latencia: Delta de tiempo To-Be vs As-Is < 15%.
ESC-06	Operabilidad (Diagnóstico)	Ocurre un fallo en una ejecución específica debido a un dato de entrada malformado.	El ingeniero identifica la causa raíz en < 2 min usando CloudWatch Logs Insights sobre el log estructurado.	MTTD: Reducción significativa del tiempo de diagnóstico manual.

Tabla 2. Escenarios Top-K

4.5 OPERACIÓN Y ESCALABILIDAD EN AWS

La etapa final del marco teórico aterriza la propuesta en condiciones operativas realistas: ejecutar n8n y el micro-framework en un entorno cloud, específicamente en Amazon Web Services (AWS), buscando una arquitectura costo-eficiente sin comprometer rendimiento, escalabilidad ni seguridad. Llevar la solución a nube obliga a revisar decisiones de diseño para aprovechar el modelo bajo demanda y servicios administrados, alineándose con buenas prácticas de computación en la nube (Amazon Web Services, Inc., 2024).

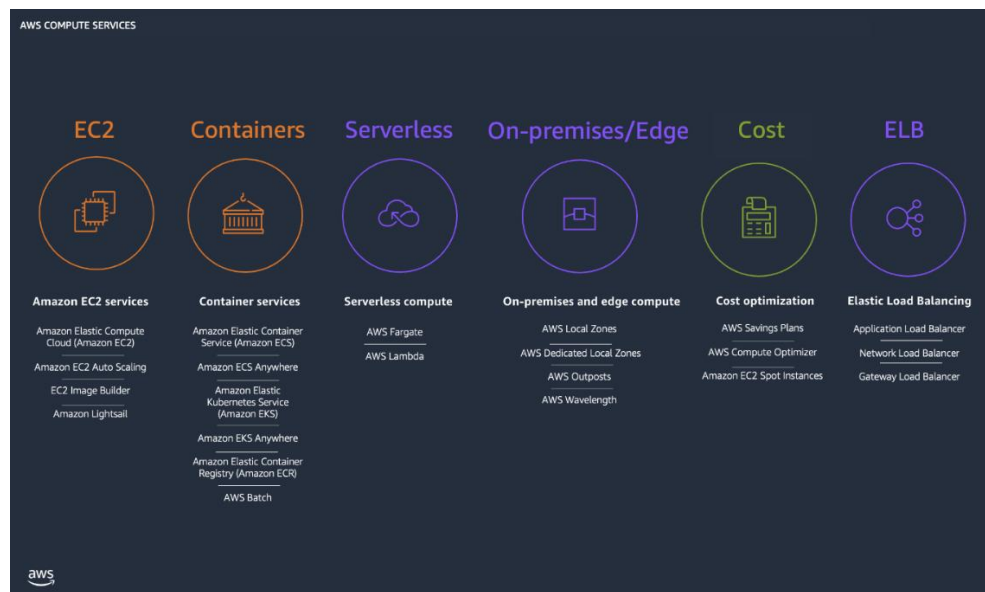
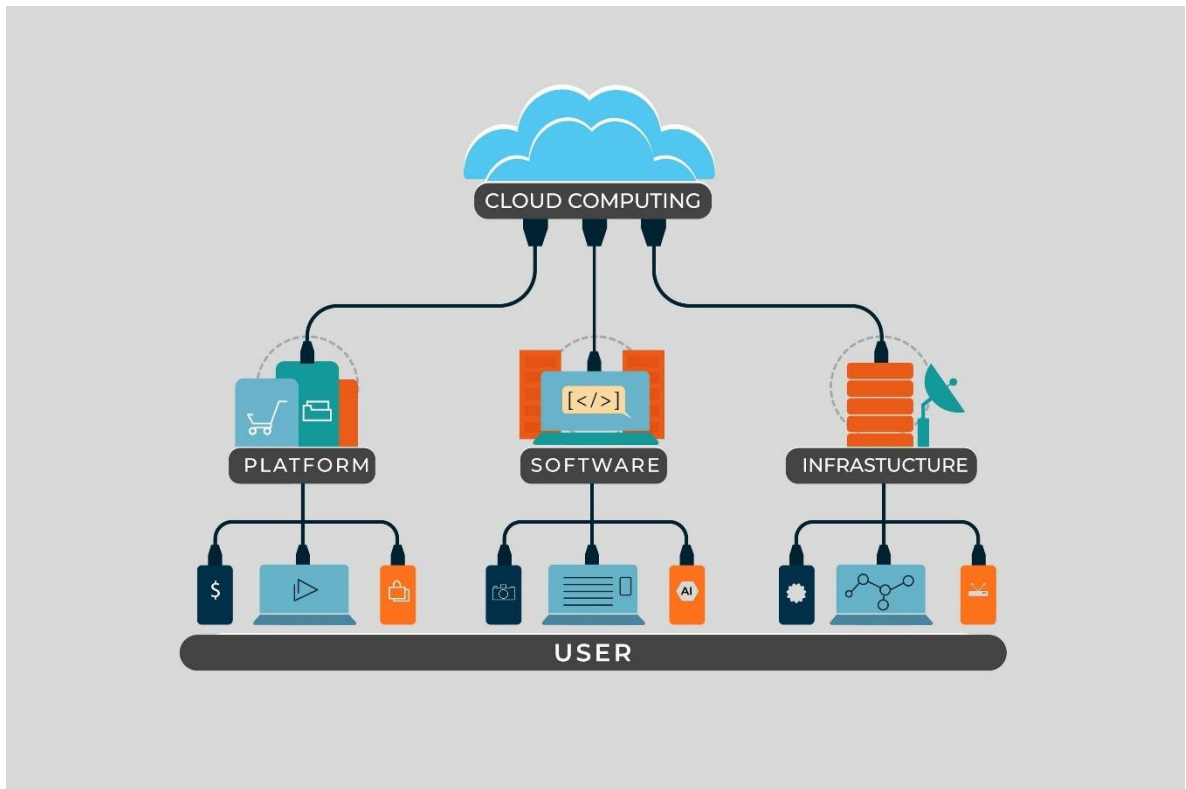


Figura 5. Diagrama de servicios de cómputo en AWS. (Fuente: AWS Whitepaper – Compute Services. Recuperado de: https://docs.aws.amazon.com/es_es/whitepapers/latest/aws-overview/compute-services.html. Consultado el 11/01/2026.)

4.5.1 Pilares de la Implementación Cloud

- Seguridad de Red e IAM: Los servicios se despliegan en una VPC bajo el principio de mínimo privilegio, utilizando roles de IAM y grupos de seguridad acotados (Amazon Web Services, Inc., 2024).
- Observabilidad Centralizada y Telemetría: Se integra Amazon CloudWatch para la recolección de métricas y logs personalizados (Amazon Web Services, 2023). Para potenciar esta capacidad, el micro-framework favorece el uso de Logging Estructurado en formato JSON. A diferencia del texto plano, este estándar permite que Amazon CloudWatch Logs Insights realice un parseo automático de campos (como request_id, execution_time o status_code), facilitando consultas complejas y la creación de tableros de control en tiempo real. Esta decisión técnica es el sustento operativo para medir las métricas de latencia y tasas de error planteadas en la metodología. (Amazon Web Services, 2023).

- Optimización de Costos: Se aplican políticas de dimensionamiento adecuado (Sizing) y gestión del ciclo de vida del almacenamiento para asegurar la viabilidad económica del proyecto (Amazon Web Services, Inc., 2024).



*Figura 6. Pilares de Implementacion Cloud
(Fuente: elaboración propia)*

La implementación de micro-frameworks en escenarios LC/NC exige un blindaje de gobernanza que asegure la calidad técnica de la solución. Esto implica integrar herramientas de CI/CD, como GitLab o Jenkins, para profesionalizar el ciclo de vida de los flujos. De igual forma, la implementación debe apoyarse en pruebas automatizadas continuas y auditorías periódicas que validen la integridad y el cumplimiento de estándares en cada proceso creado.

5. METODOLOGÍA PROPUESTA

La metodología del proyecto se plantea como un enfoque sistemático y aplicado, orientado a analizar, diseñar y evaluar un micro-framework para soluciones Low-Code/No-Code en n8n. El eje del trabajo consiste en contrastar, de forma estructurada, el estado actual de diseño y operación de los flujos (as-is) frente a una propuesta arquitectónica (to-be) basada en principios de organización por capas, estandarización de patrones y prácticas DevSecOps.

La lógica de validación se fundamenta en dos mecanismos complementarios. En primer lugar, se realiza una medición comparativa de indicadores simples y reproducibles asociados a cambios, ejecución y operación, recolectados de manera equivalente en ambos escenarios. En segundo lugar, se incorpora una evaluación arquitectónica basada en escenarios mediante ATAM para identificar riesgos, sensibilidades y trade-offs, y para explicar las diferencias observadas desde decisiones de diseño explícitas.

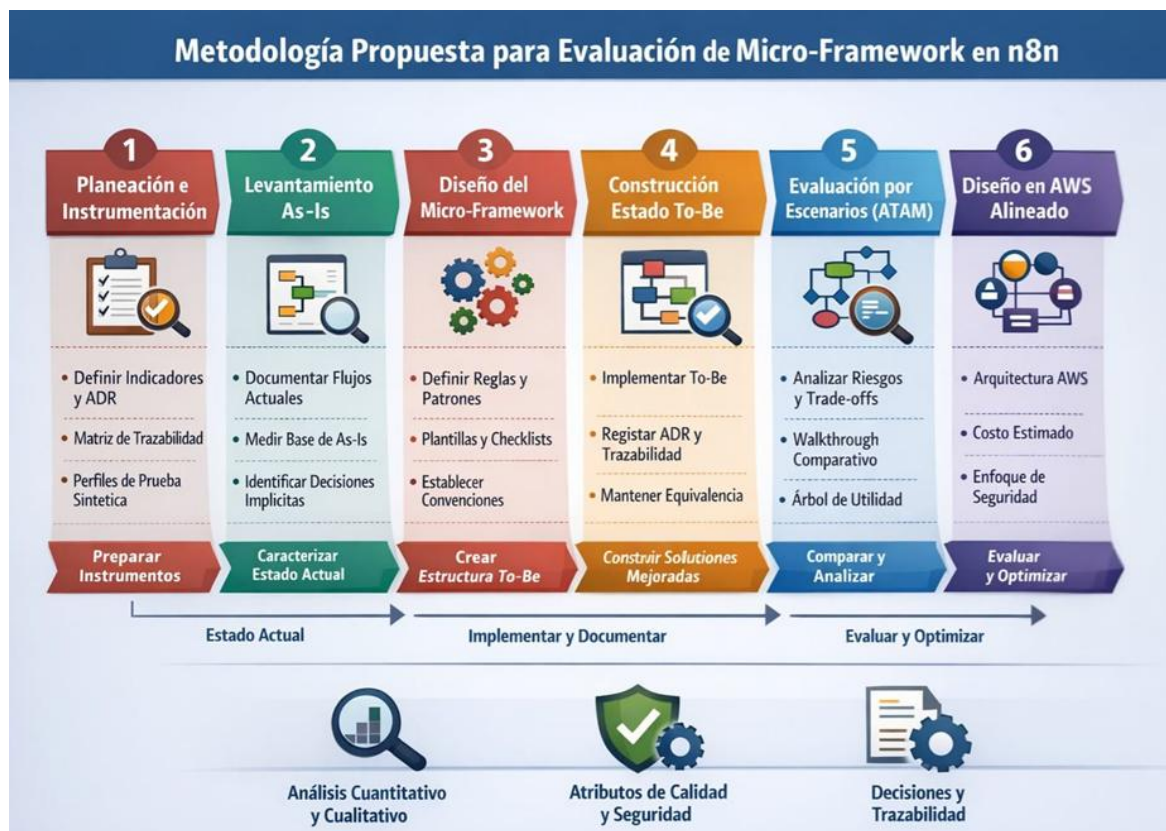


Figura 7. Metodología propuesta.
(Fuente: elaboración propia).

5.1 ENFOQUE METODOLÓGICO

El estudio adopta un enfoque de investigación mixta con secuencia explicativa. Primero se obtiene evidencia cuantitativa operativa (indicadores comparables entre as-is y to-be), y posteriormente se explica el porqué de los cambios con evidencia cualitativa basada en arquitectura y documentación (ATAM, ADR, matriz de trazabilidad y checklists).

Desde el diseño de investigación, el estudio se clasifica como cuasi-experimental bajo un esquema de comparación as-is vs to-be aplicado a dos casos de estudio. No existe asignación aleatoria ni control total del fenómeno; sin embargo, las condiciones de ejecución se mantienen controladas y replicables (misma versión, misma configuración y mismas entradas). Aun así, la intervención está claramente definida sobre el diseño, dado que el micro-framework introduce reglas, estructura y controles. Para evitar comparaciones injustas, se mantiene un criterio verificable de equivalencia funcional y un protocolo de medición uniforme para ambos estados.

En cuanto al paradigma, se asume una postura pragmática. Interesa medir diferencias observables entre versiones y, al mismo tiempo, entender cómo y por qué esas diferencias se explican por decisiones arquitectónicas (por ejemplo, separación de responsabilidades, contratos E/S, reglas de manejo de errores, gestión de secretos y observabilidad). La prioridad es que el resultado sea útil y replicable en contextos reales donde LC/NC se usa para automatización e integración.

5.2 POBLACIÓN Y MUESTRA

La población objetivo está conformada por soluciones de automatización e integración construidas en plataformas Low-Code/No-Code, en particular aquellas implementadas en n8n y usadas en contextos donde se requiere sostener atributos de calidad como mantenibilidad, confiabilidad, seguridad y trazabilidad a lo largo del tiempo. En este tipo de soluciones, el crecimiento suele darse de forma incremental y, cuando el diseño se realiza sin lineamientos de arquitectura o gobernanza, tienden a aparecer acoplamientos innecesarios, dificultades de evolución y fragilidad operativa.

La muestra se define de forma intencional, con criterios de representatividad y relevancia técnica, y se compone de dos casos de estudio que reflejan patrones comunes de adopción de LC/NC:

- Un bot o agente orientado a interacción y automatización de tareas;
- Un pipeline IoT enfocado en ingesta, validación, persistencia y notificación de datos.

Para asegurar que ambos casos sean suficientemente exigentes (sin volverlos inmanejables), la selección considera criterios mínimos como: presencia de al menos una integración externa, manejo de credenciales/secretos, necesidad de validación de entradas/salidas y requerimientos básicos de manejo de errores (reintentos, fallos controlados, trazabilidad).

El tamaño de la muestra se limita a dos casos porque el objetivo no es generalización estadística, sino un análisis comparativo con profundidad sobre decisiones arquitectónicas. Este alcance permite aplicar ATAM con el nivel de detalle requerido, documentar decisiones mediante ADR y construir matrices de trazabilidad completas, fortaleciendo la validez analítica del estudio.

5.3 TÉCNICAS E INSTRUMENTOS DE RECOLECCIÓN DE DATOS

La recolección de datos combina evidencia cuantitativa y cualitativa con un principio operativo: métricas simples, definidas por adelantado, con un protocolo de captura y evidencia verificable.

5.3.1 Definiciones operativas

Equivalencia funcional: Se considera que un flujo to-be es funcionalmente equivalente a su as-is si cumple simultáneamente:

- Procesa el mismo conjunto de entradas sintéticas (Input Set) bajo condiciones equivalentes.
- Produce salidas equivalentes según un conjunto de aserciones definidas (por ejemplo: mismos campos clave, misma estructura, mismos estados esperados, mismas notificaciones cuando aplique).
- Pasa un conjunto mínimo de casos de prueba por escenario definido.
- No introduce pasos manuales adicionales para completar el objetivo funcional (la automatización sigue siendo automatización).

Definición de tiempo de cambio: El tiempo de cambio se mide sobre cambios estándar (Change Requests) predefinidos, aplicados a ambos casos y a ambas versiones:

- CR1 (Regla de negocio): modificación de una condición o regla de decisión (por ejemplo, un umbral, una priorización, un mapeo de estados).
- CR2 (Integración): cambio de endpoint, proveedor o parámetros de integración; incluye cambios de credencial simulada (sin usar secretos reales).

- CR3 (Validación y manejo de error): incorporación o ajuste de validación de entradas/salidas y comportamiento ante fallos (reintentos, fallback, error controlado).

Un cambio se considera hecho cuando:

- Pasa el set de pruebas definido para el caso.
- El flujo queda exportado y versionado.
- Cumple el checklist mínimo aplicable (arquitectura + seguridad/operación).
- Queda registrado el commit (hash) y la evidencia del resultado.

Esto permite que el tiempo de cambio mida un proceso completo y comparable, no solo el tiempo editando nodos.

5.3.2 Componente cuantitativo: métricas y fuentes de datos

El componente cuantitativo se operacionaliza mediante un conjunto de métricas agrupadas por dimensión (entrega, operación, seguridad y trazabilidad), definidas con criterios explícitos de medición, fuente de evidencia y formato de salida comparable entre los estados as-is y to-be. La Tabla 3 resume la estructura de medición.

Dimensión	Métrica	Definición operacional	Fuente de evidencia	Unidad / Salida
Entrega	Tiempo de cambio	Minutos desde inicio CR hasta cumplimiento de pruebas + checklist + commit	CR Log + commit hash	Minutos por CR
Entrega	Impacto de cambio	# nodos/subflujos modificados por CR	Diff export + revisión manual	Conteo
Entrega	Acoplamiento práctico	# dependencias externas afectadas por CR	Checklist técnico CR	Conteo
Entrega	Re-trabajo	# intentos hasta pasar pruebas y checklist	Bitácora + commits	Conteo
Operación	Ejecuciones fallidas	Fallos / N corridas por Input Set	Run Log (status + error_type)	% fallos
Operación	Latencia tramo clave	Tiempo entre timestamp inicio-fin por tramo	Logs estructurados	ms / mediana / p95
Operación	Micro benchmark	Tiempo de respuesta bajo carga controlada	JMeter + logs	ms + tasa error
Seguridad	Exposición secretos	Presencia de secretos en nodos/export/log	Checklist + revisión export	0 / 1
Seguridad	Mínimo	Integraciones con	Checklist +	0 / 1

	privilegio	permisos estrictamente necesarios	config simulada	
Seguridad	Superficie código	# nodos con ejecución de código	Inventario flujo	Conteo
Trazabilidad	Cobertura ADR	ADR creados / decisiones esperadas	Lista decisiones vs ADR	%
Trazabilidad	Cobertura ATAM	Escenarios con evidencia trazable	Matriz escenario-evidencia	%
Trazabilidad	Cobertura checklist	Ítems cumplidos / total	Checklist binario	%

Tabla 3. Matriz de métricas cuantitativas y fuentes de evidencia

Formalización de métricas clave

Para asegurar consistencia y repetibilidad, algunas métricas se expresan mediante relaciones explícitas:

Tasa de fallos (%)

$$\text{Tasa de fallos} = \frac{\text{Ejecuciones fallidas}}{N} \times 100$$

donde N corresponde al número total de corridas por Input Set en cada versión evaluada.

Cobertura ADR (%)

$$\text{Cobertura ADR} = \frac{\text{ADR creados}}{\text{Decisiones esperadas}} \times 100$$

Cobertura ATAM (%)

$$\text{Cobertura ATAM} = \frac{\text{Escenarios con evidencia trazable}}{\text{Escenarios priorizados}} \times 100$$

Estructura de captura de datos

La medición sigue un flujo uniforme para ambos estados evaluados (*as-is* y *to-be*), garantizando comparabilidad:

- Definición de CR o Input Set.
- Ejecución controlada del flujo.
- Registro estructurado en Run Log y CR Log.
- Extracción de métricas (tiempos, diffs, errores, cumplimiento).

- Consolidación en dataset comparativo por caso y versión.

Este procedimiento asegura que todas las métricas derivan de evidencia versionada, reproducible y auditable.

Criterio de comparabilidad

Todas las métricas se calculan bajo:

- Mismos Input Sets (A/B/C).
- Mismo número mínimo de corridas ($N \geq 10$ por versión).
- Misma versión de entorno y configuración.
- Protocolo uniforme de registro.

Esto permite que las diferencias observadas puedan atribuirse principalmente a la intervención arquitectónica introducida por el micro-framework y no a variaciones externas.

5.3.3 Protocolo de captura y trazabilidad de medición

Para garantizar la trazabilidad, repetibilidad y consistencia de las mediciones, se emplearán instrumentos de registro estandarizados durante la ejecución de los casos de estudio. En particular, se utilizarán formatos de registro para documentar:

- Configuración del entorno y la versión de cada artefacto evaluado.
- Parámetros de entrada por conjunto de datos.
- Resultados observados por ejecución.
- Incidentes o fallas presentadas, incluyendo su clasificación y evidencia asociada.

Con el fin de mantener comparabilidad entre los escenarios as-is y to-be, cada corrida quedará respaldada mediante evidencias mínimas verificables (por ejemplo, registros de ejecución, capturas o reportes exportados), junto con un identificador único de corrida que permita enlazar entradas, resultados y observaciones. Los formatos de registro y los instrumentos utilizados para la recolección de evidencias (plantillas de Run Log y CR Log, junto con la taxonomía de incidentes propuesta) se presentan en el Anexo C, incluyendo un ejemplo de diligenciamiento.

Adicionalmente, se definirá un esquema de clasificación de fallos, por ejemplo, errores de entrada, errores de transformación, indisponibilidad temporal, fallos de credenciales o configuración, con el propósito de analizar su frecuencia y su impacto en el tiempo de diagnóstico y recuperación. Esta clasificación será aplicada de forma uniforme en ambos enfoques evaluados para permitir un análisis consistente.

Atributo	Métrica operacional principal	Evidencia mínima	Momento	Comparación
Mantenibilidad	Tiempo de cambio + impacto de cambio + checklist	CR Log + diff + checklist	CR1–CR3	To-be mejora vs as-is
Confiabilidad	Ejecuciones fallidas + manejo de error	Run Log + error_type + evidencia	N corridas	To-be reduce fallos
Seguridad	Exposición de secretos + mínimo privilegio	revisión export/logs + controles	pre/post	To-be reduce hallazgos
Desempeño	Latencia tareas clave (promedio/p95)	timestamps + reporte	pruebas	To-be ≤ as-is (cuando aplique)
Trazabilidad	cobertura ADR + cobertura ATAM + checklist	ADR + matriz + checklist	transversal	To-be aumenta cobertura

Tabla 4. Operacionalización resumida por atributo.

5.4 PROCEDIMIENTOS

El estudio se desarrolla por fases secuenciales, manteniendo control sobre el alcance y priorizando que cada fase produzca evidencias verificables.

5.4.1. Planeación e instrumentación

Entrada: definición de casos, alcances y atributos de calidad priorizados.
Salida: instrumentos listos, logs, checklists, plantillas y protocolo definido.
Evidencia: versión 1.0 de instrumentos en el repositorio.

Proceso:

- Definir Input Sets sintéticos (A/B/C) por caso, con semillas para reproducibilidad.
- Definir el set mínimo de pruebas por caso (criterio de equivalencia funcional).
- Definir catálogo de fallos y formatos de Run Log y CR Log.
- Construir checklist (arquitectura, seguridad, operación) con ítems binarios y evidencia requerida.
- Definir protocolos de medición de latencia (timestamps por tramo).
- Configurar repositorio y estructura de evidencias.

5.4.2. Levantamiento y caracterización as-is

Entrada: casos seleccionados, n8n local configurado y datos sintéticos.
 Salida: línea base as-is con métricas, hallazgos y artefactos.
 Evidencia: export del flujo, logs, checklist y tabla de métricas.

Proceso:

- Implementar o reconstruir el flujo as-is por caso.
- Documentar estructura (nodos, integraciones, puntos críticos).
- Ejecutar N corridas por Input Set (A/B/C) y registrar Run Logs.
- Aplicar checklist y consolidar hallazgos.
- Medir latencia por tramos definidos y consolidar métricas base.
- Para cada Input Set (A/B/C) se realizarán al menos 10 corridas ($N \geq 10$) por versión, ajustables según estabilidad observada.

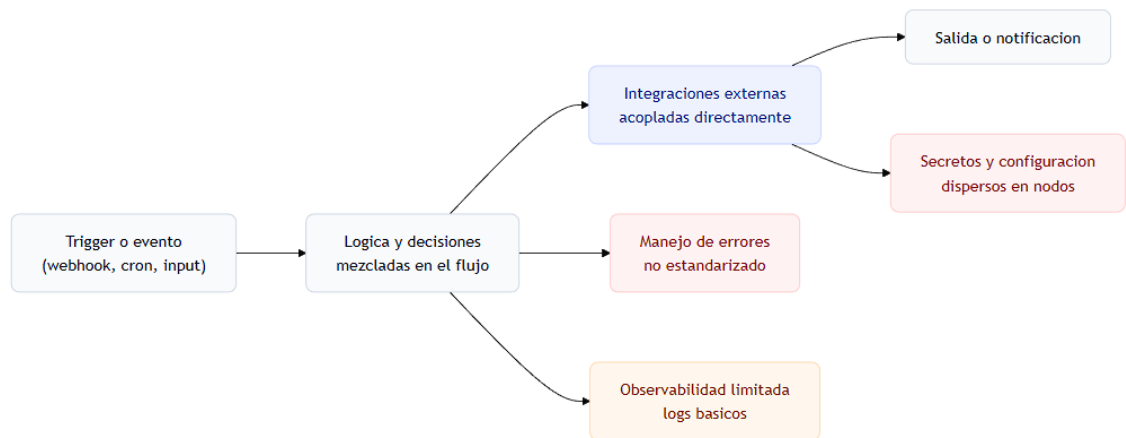


Figura 8. Diagrama as-is conceptual (Fuente: elaboración propia)

5.4.3. Diseño del micro-framework

Entrada: hallazgos as-is, atributos de calidad priorizados y restricciones del entorno.
 Salida: micro-framework definido como un conjunto verificable de entregables.
 Evidencia: documentación y plantillas versionadas.

Proceso:

- Catálogo de reglas: reglas obligatorias y recomendadas, cada una con criterio de verificación.
- Plantillas por tipo de flujo: estructura base para bot e IoT.
- Contratos de entrada y salida: esquema de payload por módulo en formato JSON.
- Convenciones de nombres: para nodos, subflujos, variables y artefactos.

- Checklist DevSecOps: mínimos de seguridad y operación.
- Guía de observabilidad: logs mínimos, campos requeridos, métricas mínimas a registrar por ejecución.

Componente	Responsabilidad	Artefacto entregable	Regla verificable
Metamodelo por etapas	Establecer estructura mínima del flujo	guía + esquema	Todo flujo declara etapa/propósito
Capas (responsabilidades)	Separar lógica y detalles de integración	reglas + ejemplos	Integraciones solo en “adaptadores”
Patrones/antipatrones	Normalizar soluciones recurrentes	catálogo	Todo flujo crítico maneja errores
Checklist de cumplimiento	Control previo	checklist binario	Umbral mínimo de cumplimiento
ADR y trazabilidad	Registrar decisiones y excepciones	plantilla ADR + matriz	Toda excepción tiene ADR
Observabilidad mínima	Logs/métricas mínimos por ejecución	guía + campos	Log estructurado por corrida

Tabla 5. Estructura técnica del micro-framework

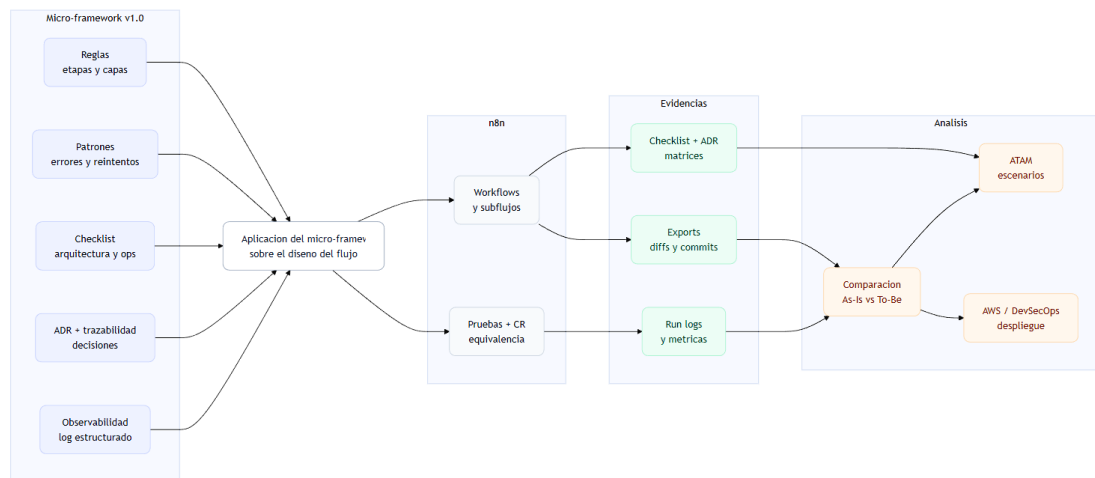


Figura 9. Diagrama conceptual micro-framework (Fuente: elaboración propia).

5.4.4. Construcción del estado to-be

Entrada: micro-framework, flujos as-is, mismos Input Sets y pruebas.
 Salida: to-be funcionalmente equivalente, evidencia de diseño y controles.
 Evidencia: export to-be, ADRs, matriz, logs y checklist.

Proceso:

- Implementar to-be aplicando reglas y plantillas.
- Registrar decisiones relevantes en ADR con estructura mínima.
- Construir la matriz de trazabilidad.
- Validar equivalencia funcional con el set de pruebas.
- Ejecutar N corridas por Input Set, capturar logs y medir latencia.
- Aplicar checklist y consolidar cumplimiento.

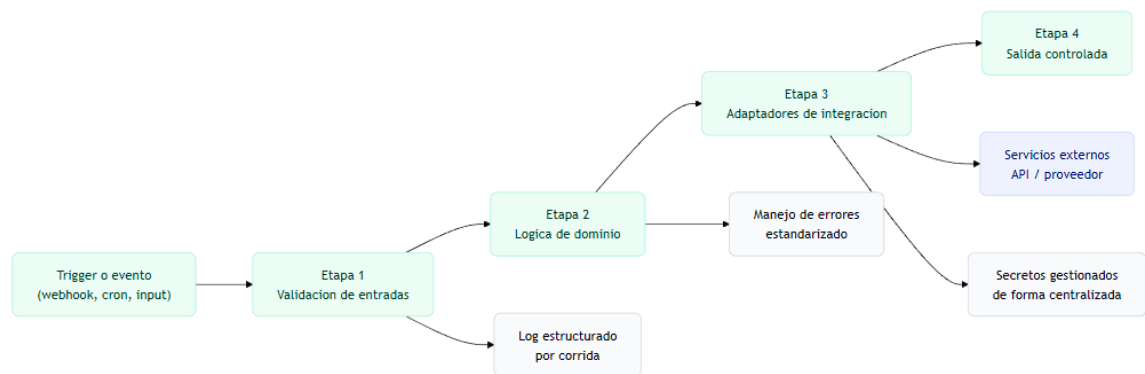


Figura 10. Diagrama To-be conceptual (Fuente: elaboración propia)

5.4.5. Evaluación arquitectónica por escenarios ATAM

La evaluación arquitectónica se hará con el método ATAM, con el objetivo de identificar y analizar los compromisos (trade-offs) entre los principales atributos de calidad del proyecto, como mantenibilidad, confiabilidad, seguridad operativa, portabilidad y desempeño. Esta evaluación se realizará con un enfoque académico y repetible, usando la evidencia obtenida en los casos de estudio y la documentación técnica del micro-framework.

Para no extender el análisis más de lo necesario, se trabajará con un conjunto reducido y priorizado de escenarios por cada caso de estudio. En específico, se seleccionarán 6 escenarios top-K por caso, definidos a partir de un árbol de utilidad y criterios de priorización como impacto y probabilidad. Cada escenario se describirá con una estructura estándar (estímulo, entorno, artefacto, respuesta y medida) y se analizará

cómo las decisiones de diseño afectan su cumplimiento. Con esto, se identificarán riesgos, no-riesgos y puntos donde existan trade-offs claros. Los resultados se organizarán en una matriz de decisiones y recomendaciones, incluyendo supuestos, limitaciones y la evidencia que los respalda.

1. La ejecución de ATAM seguirá estos pasos acotados para este caso particular:
2. Identificación de actores participantes.
3. Presentación de las arquitecturas as-is y to-be mediante diagramas y explicación de decisiones.
4. Construcción del árbol de utilidad y priorización de atributos.
5. Definición y priorización de escenarios (top-K) con estructura estímulo–respuesta–medida.
6. Walkthrough por escenario sobre as-is y to-be para identificar riesgos, sensibilidades y trade-offs.
7. Consolidación de resultados y recomendaciones vinculadas a evidencia (ADR, checklist, métricas, logs).

Finalmente, para que el análisis arquitectónico tenga coherencia con las mediciones, los hallazgos del ATAM se compararán con los resultados de la evaluación as-is/to-be (por ejemplo, tiempo de implementación, esfuerzo de diagnóstico, incidencias y repetibilidad de ejecución). Esto permitirá explicar no solo qué cambió, sino también por qué cambió, desde la perspectiva de la arquitectura del sistema.

Salida esperada ATAM:

- Lista de riesgos, puntos sensibles, trade-offs y no-riesgos.
- Matriz escenario, as-is score, to-be score y evidencia.
- Justificación del score y decisiones relacionadas.
- Relación directa con métricas observadas.

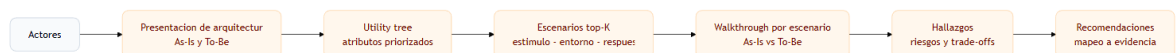


Figura 11. Resumen ATAM (Fuente: elaboración propia).

5.4.6. Diseño de arquitectura en AWS alineada

Entrada: micro-framework, requerimientos operativos y supuestos de uso.
 Salida: diseño AWS costoeficiente y escalable, alineado al micro-framework.
 Evidencia: diagramas, supuestos y estimación de costos.

Proceso:

- Definir arquitectura de referencia: servicios, red, seguridad y observabilidad.
- Especificar cómo se implementan controles DevSecOps en AWS: secretos, IAM mínimo privilegio y logging.
- Definir supuestos de carga y estimación de costos.
- Documentar decisiones cloud en ADRs si afectan trade-offs: costo vs escalabilidad vs operación.

Decisión / componente AWS	Atributo soportado	Regla asociada del micro-framework	Evidencia verificable
Gestión de secretos (servicio administrado)	Seguridad	No secretos en nodos/exports/logs	Validación + configuración
IAM mínimo privilegio (roles/políticas)	Seguridad/Gobernanza	Permisos acotados por integración	Matriz permisos + checklist
VPC/SG (segmentación y control de tráfico)	Seguridad/Resiliencia	Integraciones por endpoints permitidos	Diagrama + reglas SG
CloudWatch (logs/métricas/alertas)	Observabilidad/Confiabilidad	Log estructurado por corrida + métricas mínimas	Evidencia logs/alarms
CloudTrail (auditoría)	Seguridad/Trazabilidad	Registro de cambios/accesos	Reporte de eventos
Etiquetado + estimación de costos	Costo/Gobernanza	Recursos con tags + modelo costo	Modelo + tags

Tabla 6 Mapeo AWS, atributos, reglas, micro-frameworks y evidencias.

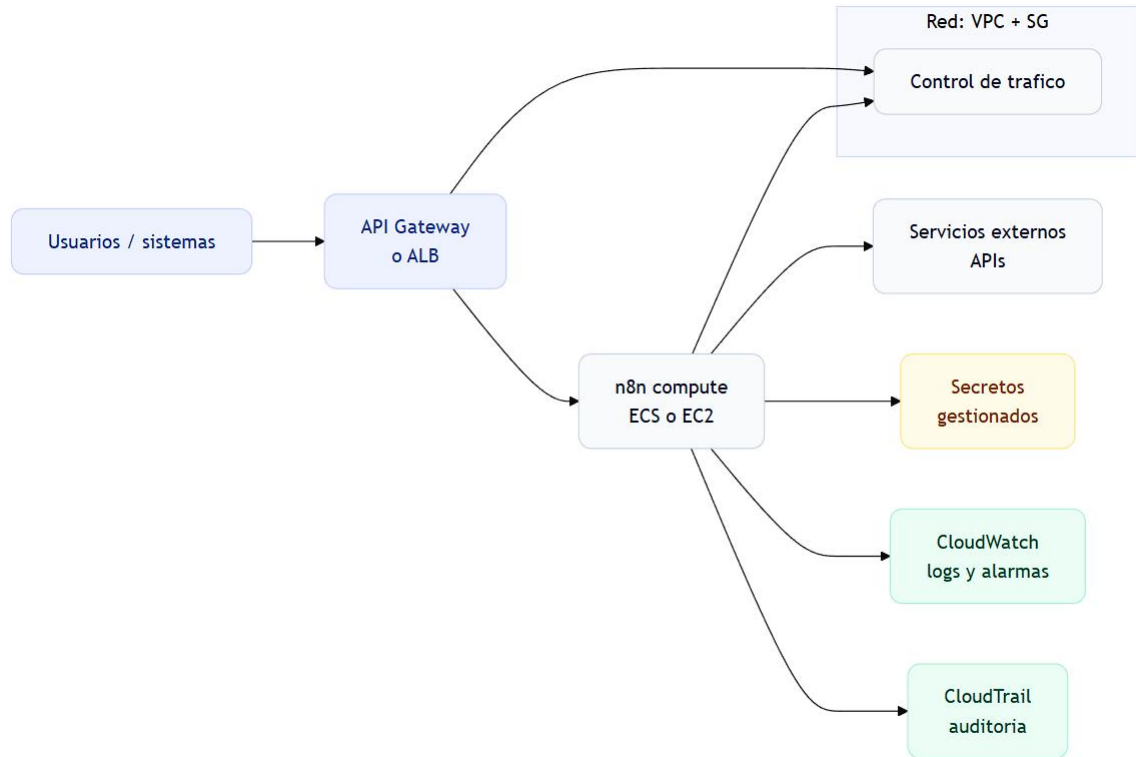


Figura 12. Diagrama de despliegue AWS de referencia.
(Fuente: elaboración propia).

5.5 ANÁLISIS DE DATOS

El análisis se realizará en dos niveles, integrando evidencia cuantitativa y cualitativa, con el fin de comparar los resultados del enfoque as-is frente al to-be en cada caso de estudio.

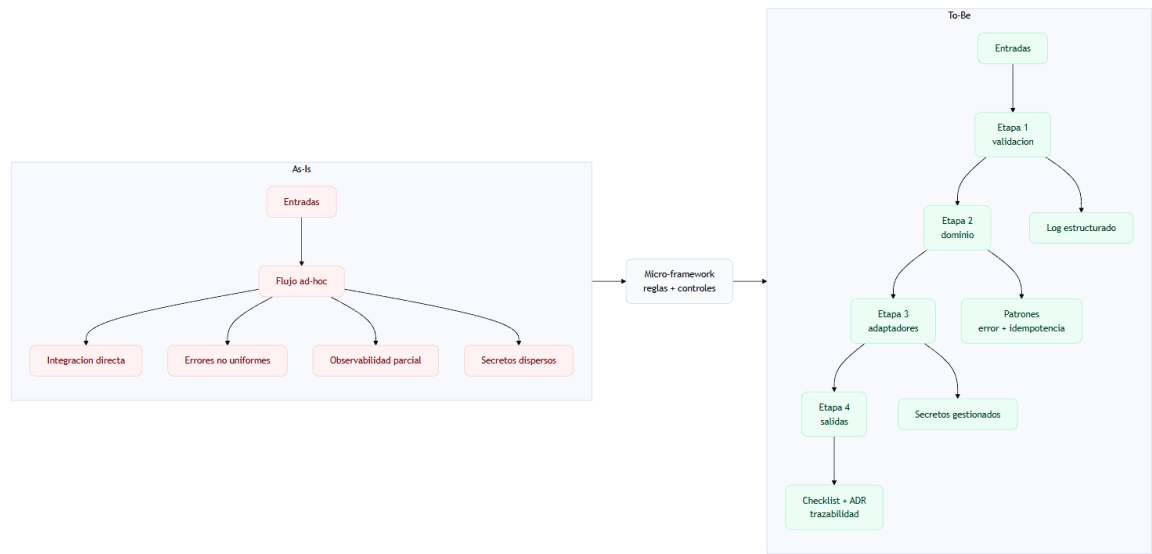


Figura 13. Comparativo as-is vs to-be (Fuente: elaboración propia).

Componente cuantitativo

Se compararán las métricas principales entre as-is y to-be, por caso de estudio y por CR (cuando aplique). Se reportarán diferencias absolutas y relativas. Cuando existan múltiples repeticiones, se presentarán medidas resumidas (por ejemplo, media o mediana) y un indicador mínimo de dispersión (por ejemplo, rango o variación necesaria) para apoyar la interpretación.

La lectura será principalmente práctica y orientada a impacto, respondiendo preguntas como:

- ¿Cuánto se reduce el tiempo de cambio y el impacto de cambio?
- ¿Cuánto disminuyen los fallos en ejecuciones controladas?
- ¿Qué ocurre con la latencia bajo condiciones de prueba acotadas?
- ¿En qué medida mejora el cumplimiento de controles definidos por checklist?

Componente cualitativo

Se analizarán de forma sistemática los artefactos generados durante el estudio (resultados de ATAM, ADRs, matriz de decisiones y checklists), con el propósito de explicar y contextualizar los resultados cuantitativos. En este análisis se buscará identificar:

- Qué reglas, decisiones o mecanismos del micro-framework explican las mejoras observadas.

- Qué trade-offs se introducen (por ejemplo, mayor estructura y estandarización a cambio de más pasos o configuraciones).
- Qué riesgos permanecen, en qué condiciones se presentan y cuál es su impacto.
- Qué elementos del framework se deben mantener, ajustar o simplificar para mejorar su aplicabilidad.

Criterios de evaluación

Para reducir la subjetividad de las conclusiones, se usarán criterios de decisión orientativos y defendibles, definidos para este estudio y aplicados en un entorno controlado de laboratorio. Estos criterios no se interpretan como garantías universales, sino como referencias para evaluar si la intervención produce mejoras consistentes y explicables en el contexto evaluado.

Se considerarán como metas orientativas las siguientes:

- Impacto de cambio: se espera una reducción aproximada de $\geq 20\%$ en el impacto de cambio en to-be frente a as-is, medido por el número de nodos/subflujos tocados por CR.
- Confiabilidad en ejecución: se espera una reducción aproximada de $\geq 30\%$ en el número de fallos durante N ejecuciones controladas por Input Set.
- Cumplimiento de controles: se espera un cumplimiento de $\geq 90\%$ del checklist en to-be.
- Evaluación ATAM: se evaluarán 6 escenarios top-K por caso (priorizados). Se espera que $\geq 80\%$ de estos escenarios cuenten con una respuesta trazable (ADR, evidencia) y que se observe una mejora en la calificación ordinal frente a as-is, cuando aplique.

En caso de que una o más metas no se alcancen, se reportarán los resultados y su interpretación, describiendo las condiciones del entorno, la evidencia disponible y los trade-offs identificados, sin invalidar el aporte del análisis.

Validez y control de sesgos

- Sesgo del investigador: el mismo ejecutor implementa as-is y to-be. Control: CR predefinidos, protocolos de medición consistentes, checklists binarios y evidencia versionada (artefactos, configuraciones y registros).
- Variación de entradas: cambios en datos sintéticos pueden alterar resultados. Control: Input Sets fijos (A/B/C), con semillas definidas y versionado de datasets para asegurar repetibilidad y comparabilidad.

6. RESULTADOS ESPERADOS

Los resultados del proyecto corresponden a productos técnicos verificables derivados del diseño, implementación y evaluación del micro-framework en n8n. Cada resultado se respalda en artefactos versionados y métricas trazables.

6.1 RESULTADOS APLICADOS

R1. Marco arquitectónico LC/NC para n8n (micro-framework) con reglas, metamodelo y guía de revisión.

Se espera entregar un micro-framework específico para n8n que relacione atributos de calidad con decisiones de diseño y prácticas operativas mínimas. Este resultado se materializa como un conjunto de artefactos aplicables y verificables, incluyendo:

Componente	Descripción
Reglas y patrones	Reglas obligatorias y recomendadas con criterio verificable
Metamodelo	Definición de componentes y límites lógicos
Plantillas base	Bot e IoT estructurados
Contratos E/S	Esquemas JSON por módulo
Convenciones	Naming para nodos y subflujos
Checklist	DevSecOps y arquitectura

Tabla 7. Componentes micro-framework

Indicador	Evidencia
Documento v1.0	Micro-framework versionado
% reglas verificables	Reglas con criterio explícito
Cobertura atributos	Regla/patrón asociado a atributo priorizado

Tabla 8. Indicadores de verificación de R1.

R2. Diseños comparativos de dos casos (as-is y to-be) con documentación completa de decisiones.

Se espera entregar dos paquetes de diseño comparativo, que incluyan el estado as-is y el estado to-be funcionalmente equivalentes, con documentación técnica que permita revisar cambios y justificar por qué se realizaron. Cada paquete incluye:

Elemento	Evidencia
Flujos exportados	Versionados

Diagramas	Arquitectura del flujo
ADR	Decisiones documentadas
Matriz trazabilidad	Requerimiento ↔ diseño
Logs	Run Log + CR Log
Dataset	Métricas consolidadas

Tabla 9. Estructura de los paquetes

Indicador	Evidencia
# ejecuciones por input_set	Run Log
CR1–CR3 aplicados	CR Log
% checklist	Checklist diligenciado
Tablas comparativas	Métricas as-is vs to-be

Tabla 10. Indicadores de verificación R2.

R3. Diseño de arquitectura en AWS costo-eficiente y escalable alineado con el micro-framework.

Se espera producir un diseño de referencia en AWS para desplegar n8n y operar flujos alineados al micro-framework. Este resultado incluye diagramas y explicación técnica de la arquitectura propuesta, especificando: componentes principales, estrategia de escalabilidad, controles básicos de seguridad y operación (gestión de secretos, mínimo privilegio, logging/monitorización), y una estimación de costo bajo supuestos explícitos de carga y retención de logs.

Elemento	Alcance
Componentes AWS	Servicios principales
Seguridad	IAM mínimo privilegio
Observabilidad	Logging y monitoreo
Escalabilidad	Estrategia progresiva
Costos	Estimación bajo supuestos explícitos

Tabla 11. Componentes del diseño

Indicador	Evidencia
Documento técnico	Diseño v1.0
Supuestos carga	Documentados
Estimación costo	Escenario mínimo

Tabla 12. Indicadores de verificación R3

6.2 RESULTADOS METODOLÓGICOS Y DE VALIDACIÓN

R4. Protocolo e informe ATAM para evaluar arquitectura en LC/NC comparando as-is vs to-be.

Se espera entregar un protocolo replicable, paso a paso, para ejecutar una evaluación tipo ATAM en soluciones LC/NC (aplicable a n8n), junto con su aplicación a los dos casos del estudio comparando as-is vs to-be.

El resultado incluye plantillas y artefactos completos: árbol de utilidad, escenarios con plantilla ATAM, esquema de priorización (Impacto, Probabilidad y Prioridad), walkthroughs comparativos, y un reporte consolidado con riesgos, trade-offs, puntos sensibles y no-riesgos. También se entregará una matriz de scoring por escenario (1–5) con evidencia trazable (ADR) y una tabla de relación explícita escenario/métrica.

Artefacto	Descripción
Utility tree	Atributos priorizados
Escenarios ATAM	Plantilla estructurada
Matriz priorización	Impacto–Probabilidad
Matriz scoring	Score 1–5 as-is/to-be
Registro riesgos	Riesgos y trade-offs

Tabla 13. Artefactos generados

Artefacto	Descripción
Utility tree	Atributos priorizados
Escenarios ATAM	Plantilla estructurada
Matriz priorización	Impacto–Probabilidad
Matriz scoring	Score 1–5 as-is/to-be
Registro riesgos	Riesgos y trade-offs

Tabla 14. Indicadores de verificación R4.

6.3 RESULTADOS DE TRANSFERENCIA Y ESTANDARIZACIÓN

R5. Guía práctica de buenas prácticas para n8n orientada a calidad, operación y adopción gradual del micro-framework.

Se espera consolidar una guía práctica que traduzca el micro-framework a recomendaciones aplicables en n8n, enfocadas en facilitar adopción, estandarización y reutilización. Incluirá prácticas mínimas y verificables sobre: validación de entrada y salida, manejo de errores y reintentos, idempotencia o control de duplicados, gestión de configuraciones y secretos, y observabilidad.

Sección	Enfoque
Validación E/S	Criterios mínimos
Manejo de errores	Reintentos e idempotencia
Seguridad	Gestión de secretos
Observabilidad	Logs estructurados
Checklist	Aplicable a flujos

Tabla 15. Contenido principal de la guía practica para n8n

6.4 RESULTADOS ACADÉMICOS

R6. Documento final del trabajo de grado.

Se espera entregar el documento final como un compendio que integre el micro-framework, la guía, los diseños as-is/to-be, la evaluación ATAM aplicada, el diseño AWS y anexos técnicos que respalden la reproducibilidad, datasets sintéticos, formatos de Run Log y CR Log, tablas de métricas, checklists diligenciados, matrices de trazabilidad, ADRs y matrices ATAM.

El documento incluirá conclusiones, limitaciones y líneas futuras, y mantendrá coherencia explícita entre objetivos, metodología, resultados y criterios de decisión.

7. ACTIVIDADES

Las actividades del proyecto se desarrollan de manera secuencial, desde la preparación del entorno y la definición del protocolo de evidencias hasta la consolidación final de resultados. En esta sección se presenta una síntesis estructurada del plan de trabajo; la descripción operativa completa de cada actividad, incluyendo técnicas, herramientas, recursos específicos y productos detallados, se encuentra en el **Anexo D**. Descripción detallada de actividades e instrumentos metodológicos. El cronograma de actividades de Gantt, que ilustra el desarrollo temporal de las tareas, se encuentra en el Anexo B. Descripción detallada de actividades e instrumentos metodológicos.

La secuencia garantiza que la comparación entre as-is y to-be se realice únicamente después de contar con condiciones controladas, instrumentos estabilizados y criterios de equivalencia funcional claramente definidos. La Tabla 16 resume cada actividad y su alcance principal.

Act.	Nombre	Enfoque técnico central	Productos principales
1	Alistamiento del entorno y protocolo de evidencias	Configuración de entorno controlado, repositorio y esquema de trazabilidad	n8n operativo, repositorio estructurado, plantillas Run Log y CR Log
2	Especificación de casos y criterios	Definición formal de casos, indicadores y datasets sintéticos	Ficha técnica por caso, criterios as-is/to-be, datasets documentados
3	Construcción del estado as-is	Implementación y documentación estructural de la línea base	Paquete as-is (flujo exportado, diagramas, notas técnicas)
4	Diseño del micro-framework	Definición de reglas, patrones y DevSecOps	Micro-framework v1.0 + plantillas
5	Construcción del estado to-be	Rediseño estructurado aplicando reglas del marco	Paquete to-be + ADR + evidencia checklist
6	Prueba piloto de instrumentos	Validación y ajuste de indicadores y formatos de registro	Instrumentos estabilizados + reporte piloto
7	Medición comparativa	Ejecuciones controladas y consolidación de métricas as-is/to-be	Dataset comparativo + evidencia CR1–CR3
8	Evaluación ATAM	Análisis de escenarios, riesgos y trade-offs	Utility tree + matriz de hallazgos
9	Diseño arquitectura AWS	Propuesta cloud alineada a atributos de calidad	Diagrama AWS + estimación de costos
10	Consolidación y cierre	Sistematización de buenas prácticas y documentación	Guía v1.0 + documento final completo

		final	
--	--	-------	--

Tabla 16. Resumen actividades principales

8. RECURSOS NECESARIOS

Los recursos necesarios para ejecutar el proyecto se definen en función del alcance técnico y metodológico planteado. La ejecución se desarrollará de forma remota, empleando herramientas habituales en ingeniería de software, documentación técnica y evaluación en entorno controlado.

Dado que el estudio se realiza con datos sintéticos y sin despliegue productivo real, no se requiere infraestructura empresarial ni servicios cloud operativos. Los recursos se organizan en cuatro categorías: computacionales, herramientas especializadas, infraestructura conceptual en la nube y soporte académico. El detalle del presupuesto requerido para estas actividades se encuentra en el Anexo A.

8.1 RECURSOS COMPUTACIONALES

El proyecto requiere un entorno de trabajo capaz de ejecutar n8n en local, realizar pruebas sintéticas y mantener trazabilidad mediante control de versiones.

Componente	Requerimiento
Procesador	4–8 núcleos
Memoria RAM	≥ 16 GB
Almacenamiento	SSD ≥ 512 GB
Sistema operativo	Windows (WSL2), Linux o macOS
Contenerización	Docker / Docker Compose

Tabla 17. Especificaciones mínimas computacionales.

El control de versiones se realizará mediante repositorio Git remoto, permitiendo trazabilidad de flujos exportados, plantillas, instrumentos y evidencias no sensibles. Se contempla respaldo periódico en repositorio remoto y almacenamiento complementario según disponibilidad.

Las credenciales y variables sensibles se gestionarán fuera del repositorio mediante configuraciones locales controladas.

8.2 SOFTWARE ESPECIALIZADO Y HERRAMIENTAS DE SOPORTE

El proyecto utiliza herramientas consolidadas en entornos de desarrollo y pruebas, garantizando reproducibilidad y bajo costo.

Categoría	Herramienta	Propósito
Automatización	n8n	Construcción y exportación de flujos

Entorno reproducible	Docker	Despliegue controlado
Editor	Visual Studio Code	Desarrollo y revisión
Pruebas	Apache JMeter	Carga sintética comparativa
Modelado	draw.io	Diagramas as-is, to-be y AWS
Documentación	Microsoft Word	Documento y anexos
Versionamiento	GitHub	Trazabilidad
Análisis de métricas	Excel / Sheets (Python opcional)	Consolidación de indicadores

Tabla 18. Herramientas necesarias para el proyecto.

8.3 INFRAESTRUCTURA Y SERVICIOS EN LA NUBE

El diseño de arquitectura en AWS se realiza con fines de referencia técnica y estimación de costos, sin despliegue productivo obligatorio.

Se utilizará documentación oficial de AWS y la AWS Pricing Calculator para estimar escenarios bajo supuestos explícitos de carga, concurrencia y retención de logs.

Los servicios considerados en el diseño incluyen IAM (identidades y permisos), almacenamiento (S3), observabilidad (CloudWatch) y componentes de cómputo acordes al esquema propuesto, tomando como referencia el AWS Well-Architected Framework.

En caso de validación técnica puntual, se podrá utilizar una cuenta AWS no productiva.

8.4 RECURSOS DE SOPORTE Y CONECTIVIDAD

Para el proyecto requiere conectividad estable para acceso a repositorios, bibliografía académica y acompañamiento institucional. Asimismo, se emplearán datasets sintéticos versionados (CSV/JSON) para garantizar repetibilidad y evitar el uso de información sensible.

El recurso humano principal es el investigador, responsable de la ejecución técnica y consolidación de entregables, con acompañamiento académico del director del trabajo de grado.

BIBLIOGRAFÍA

Ahmeti, B., Linder, M., Groner, R., & Wohlrab, R. (2024). Architecture Decision Records in Practice: An Action Research Study. In M. Galster, P. Scandurra, T. Mikkonen, P. Oliveira Antonino, E. Y. Nakagawa, & E. Navarro (Eds.), *Software Architecture (ECSA 2024)* (Lecture Notes in Computer Science, Vol. 14889, pp. 333–349). Springer. https://doi.org/10.1007/978-3-031-70797-1_22

Ajimati, M. O., Carroll, N., & Maher, M. (2025). Adoption of low-code and no-code development: A systematic literature review and future research agenda. *Journal of Systems and Software*, 222, 112300. <https://doi.org/10.1016/j.jss.2024.112300>

Amazon Web Services. (2023, July 31). Implementing Microservices on AWS (AWS Whitepaper). <https://docs.aws.amazon.com/pdfs/whitepapers/latest/microservices-on-aws/microservices-on-aws.pdf>

Amazon Web Services. (2024, November 6). Framework Well-Architected da AWS (AWS Well-Architected Framework) [PDF]. https://docs.aws.amazon.com/pt_br/wellarchitected/latest/framework/wellarchitected-framework.pdf

Binzer, B., Elshan, E., Fürstenau, D., & Winkler, T. J. (2024). Establishing a Low-Code/No-Code-Enabled Citizen Development Strategy. *MIS Quarterly Executive*, 23(3), Article 3. <https://aisel.aisnet.org/misqe/vol23/iss3/3>

Bratincevic, J., Taylor, R., & Stone, Z. (2024, January 29). The low-code market could approach \$50 billion by 2028. Forrester. <https://www.forrester.com/blogs/the-low-code-market-could-approach-50-billion-by-2028/>

Docker, Inc. (n.d.). *n8nio/n8n* [Docker image]. Docker Hub. Retrieved January 12, 2026, from <https://hub.docker.com/r/n8nio/n8n>

Feio, C., Santos, N., Escravana, N., & Pacheco, B. (2024). An empirical study of DevSecOps focused on continuous security testing. In *2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)* (pp. 608–615). IEEE. <https://doi.org/10.1109/EuroSPW61312.2024.00074>

Fortune Business Insights. (2024). Low code development platform market size, share & industry analysis, 2024–2032. <https://www.fortunebusinessinsights.com/low-code-development-platform-market-102972>

Gartner, Inc. (2021, November 10). Gartner says cloud will be the centerpiece of new digital experiences. <https://www.gartner.com/en/newsroom/press-releases/2021-11-10-gartner-says-cloud-will-be-the-centerpiece-of-new-digital-experiences>

Gartner, Inc. (2022, December 13). Gartner forecasts worldwide low-code development technologies market to grow 20% in 2023. <https://www.gartner.com/en/newsroom/press-releases/2022-12-13-gartner-forecasts-worldwide-low-code-development-technologies-market-to-grow-20-percent-in-2023>

GitHub. (2025). CVE-2025-68668: n8n vulnerable to arbitrary command execution in Pyodide based Python Code Node (GHSA-62r4-hw23-cc8v). GitHub Advisory Database. <https://github.com/advisories/GHSA-62r4-hw23-cc8v>

Gomes, D., Rego, A., & Trinta, F. (2025). Observability of microservices-based applications: A systematic mapping study. *Computing*, 107, Article 183. <https://doi.org/10.1007/s00607-025-01540-w>

Highland Europe. (2025). n8n raises €55 million from Highland Europe to transform workflow automation for technical teams by unifying AI, code and human building blocks. <https://www.highlandeurope.com/n8n-raises-e55-million-from-highland-europe-to-transform-workflow-automation-for-technical-teams-by-unifying-ai-code-and-human-building-blocks/>

Hoogsteen, D., & Borgman, H. (2022). *Empower the workforce, empower the company? Citizen development adoption*. In *Proceedings of the 55th Hawaii International Conference on System Sciences (HICSS 2022)* (pp. 4717–4726). <https://pure.uva.nl/ws/files/121260540/0463.pdf>

International Organization for Standardization. (2011). *ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. ISO.

International Organization for Standardization. (2024). *ISO/IEC 25002: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Quality model overview and usage*. ISO.

International Organization for Standardization & International Electrotechnical Commission. (2023). ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model (Edition 2). ISO. <https://www.iso.org/standard/78176.html>

Kazman, R., Klein, M. H., & Clements, P. C. (2000). ATAM: Method for Architecture Evaluation (Technical Report CMU/SEI-2000-TR-004). Software Engineering Institute, Carnegie Mellon University. <https://www.sei.cmu.edu/library/atam-method-for-architecture-evaluation/>

Lin, Y.-W., Lin, Y.-B., Wu, Y.-F., & Shen, P.-H. (2025). VoiceTalk: A no-code approach for creating voice-controlled smart home applications. *IEEE Open Journal of the Computer Society*, 6, 874–883. <https://doi.org/10.1109/OJCS.2025.3576725>

Lomonaco, V., et al. (2023). AI-Toolkit: A microservices architecture for low-code decentralized machine intelligence. In *2023 IEEE International Conference on Acoustics, Speech, and Signal Processing Workshops (ICASSPW)* (pp. 1–5). IEEE. <https://doi.org/10.1109/ICASSPW59220.2023.10193222>

Martin, R. C. (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Pearson.

Mendix. (2025, March 28). A survey of the low-code market: The Low-Code Perspective. <https://www.mendix.com/blog/low-code-market/>

Microsoft. (2025). Security and governance considerations in Power Platform. Microsoft Learn. <https://learn.microsoft.com/en-us/power-platform/admin/governance-considerations>

n8n. (2025). n8n closes €55M Series B round led by Highland Europe. <https://blog.n8n.io/series-b/>

National Institute of Standards and Technology. (2025). NVD—CVE-2025-68668. National Vulnerability Database. <https://nvd.nist.gov/vuln/detail/CVE-2025-68668>

Prates, K., & Pereira, T. (2025). DevSecOps practices and tools: A multivocal literature review. *International Journal of Information Security*, 24, Article 11. <https://doi.org/10.1007/s10207-024-00914-z>

Rajapakse, D. C., Zahedi, M., Babar, M. A., & Shen, H. (2022). Challenges and solutions when adopting DevSecOps: A systematic review and future research agenda. *Information and Software Technology*, 147, 106888. <https://doi.org/10.1016/j.infsof.2021.106700>

Santiago-Salazar, J. A., & Rico-Bautista, D. (2024). Clean Architecture: Impact on Performance and Maintainability of Native Android Projects. In M. Tabares, P. Vallejo, B. Suarez, M. Suarez, O. Ruiz, & J. Aguilar (Eds.), *Advances in Computing (CCC 2023)* (Communications in Computer and Information Science, Vol. 1924, pp. 82–90). Springer. https://doi.org/10.1007/978-3-031-47372-2_8

Smuts, S., & Smuts, H. (2022). Society 5.0 and the future of work skills for software engineers and developers. In *Proceedings of the Society 5.0 Conference 2022 – Integrating Digital World and Real World to Resolve Challenges in Business and Society* (EPIc Series in Computing, Vol. 84, pp. 169–182). EasyChair. <https://easychair.org/publications/paper/tXG6>

Souppaya, M., Scarfone, K., & Dodson, D. (2022). Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities (NIST Special Publication 800-218). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-218>

Trieflinger, S., Petrik, D., Polat, E., & Roling, B. (2025). Potentials and Risks of the Low-Code Development: A Systematic Literature Review. In M. Möhring et al. (Eds.), *Herman Hollerith Conference 2024* (Informatik aktuell, pp. 63–73). Springer. https://doi.org/10.1007/978-3-658-48215-2_8

Viljoen, A., Radić, M., Hein, A., Nguyen, J., & Krcmar, H. (2024). Governing Citizen Development to Address Low-Code Platform Challenges. *MIS Quarterly Executive*, 23(3), Article 6. <https://aisel.aisnet.org/misqe/vol23/iss3/6/>

Wan, M., & Wang, H. (2024). Streamlining data exchange in intelligent mines: Cross-system services leveraging the industrial internet platform. In *2024 9th*

International Conference on Cloud Computing and Big Data Analytics (ICCCBDA) (pp. 301–305). IEEE. <https://doi.org/10.1109/ICCCBDA61447.2024.10569246>

Wang, S.-K., Ma, S.-P., Lai, G.-H., & Chao, C.-H. (2024). ChatOps for microservice systems: A low-code approach using service composition and large language models. *Future Generation Computer Systems*, 161, 518–530. <https://doi.org/10.1016/j.future.2024.07.029>

Zhu, H., & Cai, H. (2024). Low-coupling and high-cohesion microservice partitioning method to support rapid development of steel industry management systems. In 2024 6th International Conference on Computer Communication and the Internet (ICCCI) (pp. 78–82). IEEE. <https://doi.org/10.1109/ICCCI62159.2024.10674297>

ANEXOS

ANEXO A: PRESUPUESTO

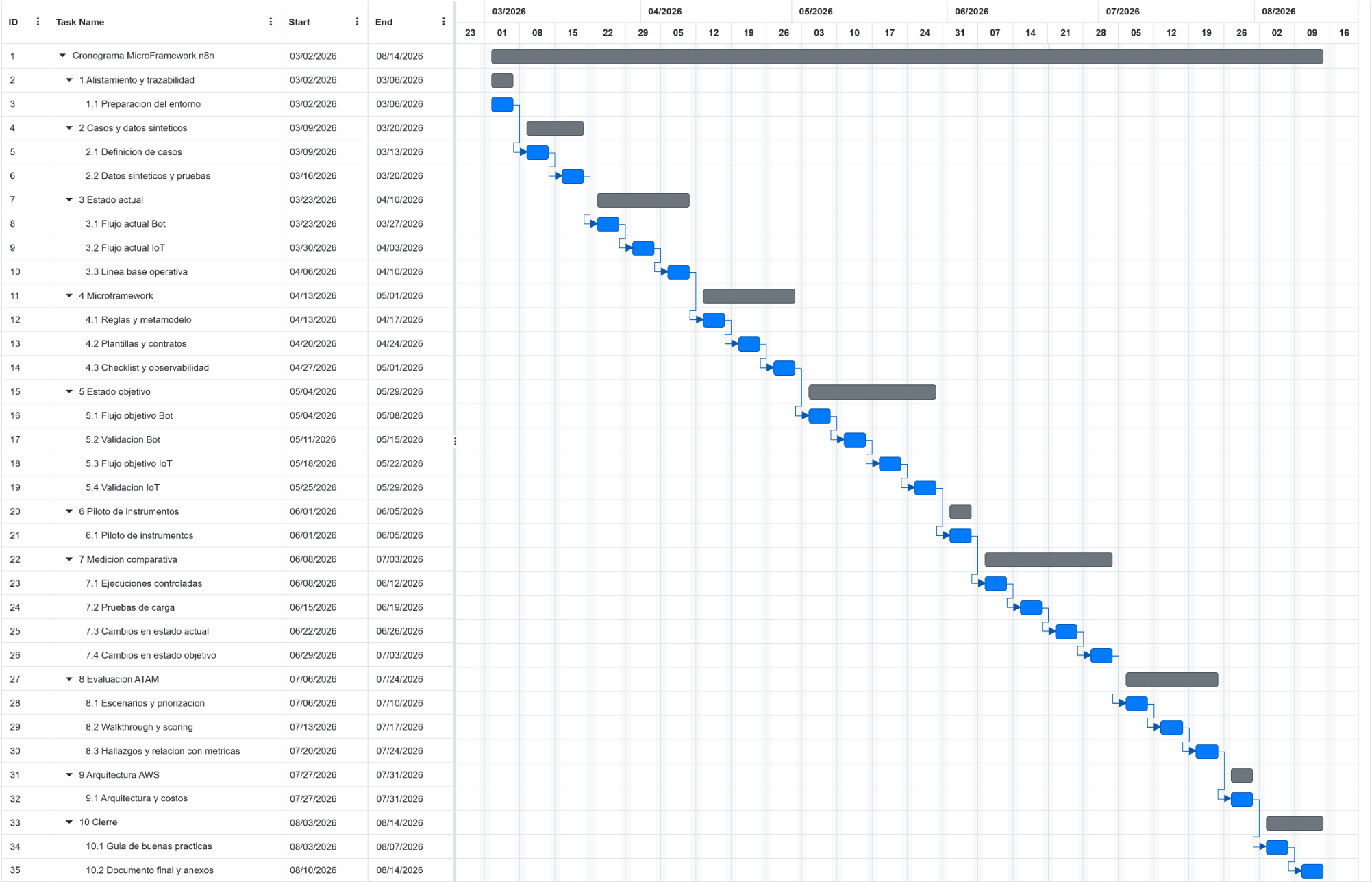
El presupuesto reúne los recursos necesarios para ejecutar el proyecto según la metodología, las actividades y el cronograma. Incluye costos de talento humano investigador y director, además de recursos materiales y tecnológicos como equipo de cómputo, dispositivos y herramientas de software. Cada valor se estima para el periodo total del proyecto. También se listan los recursos ya disponibles o de uso gratuito con costo \$0, para dejar evidencia de su uso y su aporte a la ejecución, aunque no impliquen un pago adicional.

Categoría	Descripción	Unidad	Cantidad	Valor unitario (COP)	Valor total (COP)
Recursos humanos	Investigador principal (dedicación 16 h/semana × 24 semanas)	Hora	384	37.500	14.400.000
Recursos humanos	Director del proyecto (máx. 2 h/semana × 24 semanas)	Hora	48	250.000	12.000.000
Recursos computacionales	Equipo de cómputo principal	Unidad	1	4.000.000	4.000.000
Recursos computacionales	Almacenamiento y respaldo	Servicio	1	0	0
Comunicación	Conectividad a internet para trabajo remoto	Servicio	1	0	0
Software y soporte	Python (uso local)	Licencia	1	0	0
Software y soporte	Docker y Docker	Licencia	1	0	0

		Compose				
Software soporte	y	Apache JMeter	Licencia	1	0	0
Software soporte	y	Herramienta de diagramación draw.io	Licencia	1	0	0
Software soporte	y	Control de versiones (GitHub)	Servicio	1	0	0
Software soporte	y	Hojas de cálculo (Excel o Google Sheets)	Licencia/Servicio	1	0	0
Software soporte	y	Python (opcional)	Licencia	1	0	0
Nube		AWS (diseño de referencia y estimación de costos; sin despliegue productivo)	Servicio	1	0	0
Datos de prueba	de	Datasets sintéticos (generación local, insumo presupuestado)	Conjunto	1	0	0
Otros		Espacio de trabajo remoto y energía	Servicio	1	0	0
		TOTAL GENERAL				30.900.000

ANEXO B: CRONOGRAMA DE ACTIVIDADES

El cronograma define una ruta de trabajo ordenada y verificable, coherente con la metodología y con los resultados esperados. Se estructura en diez actividades principales que avanzan desde el alistamiento del entorno y el esquema de evidencias, hasta la construcción y comparación de los estados as-is y to-be, la medición de métricas, la evaluación arquitectónica por escenarios con ATAM, el diseño de referencia en AWS y el cierre con la guía y el documento fin



ANEXO C: FORMATOS DE REGISTRO

Formato mínimo de registro por ejecución (Run Log)

Campo	Descripción	Tipo	Ejemplo
run_id	Identificador único de la ejecución para trazabilidad y correlación.	String (UUID o consecutivo)	RUN-20260115-0007, b3f1c2...
case	Caso de estudio al que pertenece la ejecución.	Enum	bot, iot
version	Versión evaluada del flujo.	Enum	as-is, to-be
input_set	Conjunto de datos sintéticos usado en la corrida.	Enum	A, B, C
start_ts	Timestamp de inicio de ejecución.	Datetime ISO 8601	2026-01-15T10:21:33-05:00
end_ts	Timestamp de fin de ejecución.	Datetime ISO 8601	2026-01-15T10:21:41-05:00
status	Resultado global de la ejecución.	Enum	success, fail
error_type	Clasificación simple del error cuando status=fail.	Enum (catálogo)	validation, integration, logic, timeout, unknown, N/A
notes	Observaciones breves de contexto (qué pasó, qué se intentó).	String (texto corto)	Falla API 503, Campo temperature missing
commit_hash	Referencia al commit que contiene el flujo/export asociado a esa ejecución.	String (hash Git)	a1b2c3d, 9f81d4e... (o N/A)

Catálogo de error_type (Run Log)

error_type	Descripción operativa	Criterio rápido de uso	Ejemplo típico
validation	Fallo por datos inválidos o incompletos antes de ejecutar integraciones críticas.	El flujo detecta que faltan campos, tipos incorrectos o valores fuera de rango.	Falta user_id, payload sin estructura
integration	Fallo originado en servicios externos o conectores (API, DB, webhook).	La falla aparece al llamar un servicio externo o al recibir respuesta no válida.	401/403, 5xx, DNS, conexión caída
logic	Fallo por lógica interna del flujo (reglas, transformaciones, condiciones).	El flujo falla aun con integraciones OK; error en ramas, mappings, cálculos.	condición mal definida, null pointer en nodo de código
timeout	Fallo por exceder tiempos límite definidos.	Expira una espera, request o ejecución.	request timeout, job tarda demasiado
unknown	Fallo no clasificable con lo anterior o sin evidencia suficiente.	No hay información suficiente para ubicarlo; se marca y se investiga.	log incompleto, error genérico

Formato mínimo de registro por Change Request (CR Log).

Campo	Descripción	Tipo formato sugerido	Valores posibles / ejemplo
cr_id	Tipo de cambio estándar aplicado.	Enum	CR1, CR2, CR3
case	Caso de estudio afectado por el cambio.	Enum	bot, iot
version	Versión sobre la que se aplica el cambio.	Enum	as-is, to-be
start_ts	Timestamp de inicio del	Datetime ISO	2026-01-

	trabajo del cambio.	8601	15T14:00:00-05:00
end_ts	Timestamp de finalización del cambio.	Datetime ISO 8601	2026-01-15T15:12:00-05:00
done_criteria	Criterios para declarar el cambio “hecho”.	Lista/Texto estructurado	tests_passed + export_versioned + checklist_min_ok
nodes_touched	Cantidad de nodos/subflujos modificados por el cambio.	Integer	3, 12
deps_touched	Cantidad de dependencias externas afectadas (APIs, credenciales, servicios).	Integer	0, 1, 2
attempts	Número de intentos/iteraciones hasta cumplir done_criteria.	Integer	1, 4
commit_hash	Commit donde queda el cambio aplicado y versionado.	String (hash Git)	c9d2a10, fe12bc...
evidence_links	Rutas en el repositorio a export/logs/checklist asociados al cambio.	Lista de strings (paths)	["/flows/to-be/bot/v2.json", "/checklists/bot/to-be_v2.md"]

Plantilla de escenario ATAM

Campo	Qué significa	Guía para llenarlo	Ejemplo corto
Estímulo	Evento o cambio que “pone a prueba” el sistema.	Describe el evento en una frase.	“Cambia la regla de aprobación.”
Fuente	Quién o qué genera el	Actor interno/externo.	“Área de negocio”, “API

	estímulo.		externa", "Sensor IoT".
Artefacto	Parte del sistema afectada por el estímulo.	Módulo/subflujo/nodo o integración.	"Módulo de decisión", "Adaptador API", "Persistencia".
Entorno	Condiciones bajo las cuales ocurre el estímulo.	Normal, degradado, con carga, post-despliegue, etc.	"Ejecución normal", "Carga media", "Servicio externo inestable".
Respuesta	Qué debe hacer el sistema idealmente.	Comportamiento esperado y controlado.	"Aplica la regla sin tocar integraciones; registra decisión."
Medida	Cómo se verifica que la respuesta es aceptable.	Métrica o criterio verificable.	"≤2 nodos tocados; pasa pruebas; logs con run_id."

Escala de priorización de Impacto para ATAM.

Valor	Interpretación operativa	Señales típicas
1	Impacto bajo	No afecta flujo principal, no genera fallos ni costos relevantes.
2	Impacto moderado	Afecta una parte menor, puede requerir ajuste pequeño.
3	Impacto medio	Afecta ejecución normal o mantenimiento frecuente.
4	Impacto alto	Puede detener operación o causar errores visibles/ <u>recurrentes</u> .
5	Impacto crítico	Riesgo de caída, pérdida de datos, incumplimiento o exposición de seguridad.

Escala de priorización de Probabilidad para ATAM.

Valor	Interpretación operativa	Señales típicas
1	Raro	Solo en condiciones excepcionales.
2	Poco frecuente	Ocurre ocasionalmente (pocas veces al año).
3	Posible	Ocurre a veces (mensual).
4	Frecuente	Ocurre de forma regular (semanal).
5	Muy frecuente	Es parte del día a día (diario o muy probable).

ANEXO D: DETALLE ACTIVIDADES

Actividad 1. Alistamiento del entorno y protocolo de evidencias

Se prepara el entorno de trabajo para garantizar trazabilidad desde el inicio. Esto incluye configurar una instancia controlada de n8n, organizar un repositorio con control de versiones y definir un protocolo mínimo para registrar evidencias, decisiones y cambios durante el desarrollo de as-is y to-be.

- Técnicas y herramientas: n8n local, Github, Visual Studio Code, plantillas base (ADR, trazabilidad, checklist), herramienta de diagramación draw.io o similar.
- Responsable: Elian Hernando Gil Sierra.
- Recursos: computador, internet, almacenamiento para repositorio, herramientas ofimáticas y de diagramación.
- Productos: entorno operativo de n8n, repositorio estructurado, protocolo de evidencias y trazabilidad, plantillas y formatos mínimos de Run Log y CR Log definidos y versionados.

Actividad 2. Especificación operativa de casos, criterios de comparación y datos sintéticos

Se definen formalmente los dos casos (bot/agente y pipeline IoT) con funcionalidad mínima y criterios de aceptación, para asegurar que as-is y to-be sean equivalentes en lo funcional. También se establecen indicadores y reglas de registro para medir de forma consistente. Además, se construyen datasets sintéticos por caso y perfiles de prueba de baja y media carga, evitando datos sensibles.

- Técnicas y herramientas: plantilla de requerimientos mínimos, criterios de aceptación, matriz de indicadores y definiciones operativas, datasets sintéticos (CSV/JSON), editor, Apache JMeter.
- Responsable: Elian Hernando Gil Sierra.
- Recursos: computador, herramientas ofimáticas, acceso a n8n, almacenamiento para datasets y evidencias.
- Productos: ficha técnica por caso, criterios de comparación as-is/to-be, indicadores operativos, dataset sintético y perfiles de prueba documentados.

Actividad 3. Construcción y documentación del estado actual (as-is)

Se implementan o reconstruyen los flujos as-is de ambos casos para establecer una línea base verificable. Se documentan estructura, dependencias, puntos de acoplamiento, validaciones, manejo de errores y consideraciones operativas, apoyándose en diagramas y notas técnicas.

- Técnicas y herramientas: n8n, exportación de flujos, diagramas as-is, notas técnicas, repositorio versionado.
- Responsable: Elian Hernando Gil Sierra.
- Recursos: computador, n8n, diagramación, repositorio, datos de prueba controlados.
- Productos: paquete as-is por caso (flujo exportado + diagramas + notas técnicas).

Actividad 4. Diseño del micro-framework LC/NC para n8n (v1.0)

Se diseña el micro-framework como un marco aplicable a flujos en n8n, con reglas, convenciones y patrones orientados a mejorar atributos de calidad sin perder agilidad. Se incorporan elementos de documentación y control (ADR, trazabilidad, checklist) y lineamientos básicos de seguridad y operación DevSecOps.

- Técnicas y herramientas: documento del micro-framework, matriz calidad/decisiones, metamodelo, reglas (capas, límites, dependencias, naming), catálogo de patrones/antipatrones, checklist, plantillas (ADR y trazabilidad).
- Responsable: Elian Hernando Gil Sierra.
- Recursos: computador, editor, repositorio, diagramación, material de soporte del proyecto.
- Productos: micro-framework v1.0 + plantillas listas para uso.

Actividad 5. Construcción del estado propuesto (to-be) aplicando el micro-framework

Se rediseñan e implementan los flujos to-be para ambos casos, conservando equivalencia funcional con as-is. Se aplican de forma explícita reglas y patrones del micro-framework, reforzando estructura, validaciones, manejo de errores, reintentos e idempotencia cuando aplique, además de prácticas básicas de seguridad y operación. Se documentan decisiones y se consolida la trazabilidad.

- Técnicas y herramientas: n8n, exportación de flujos, diagramas to be, ADR, matriz de trazabilidad, checklist de revisión.
- Responsable: Elian Hernando Gil Sierra.
- Recursos: computador, n8n, diagramación, repositorio, plantillas del micro-framework.
- Productos: paquete to-be por caso (flujo exportado, diagramas, ADR, trazabilidad) y evidencia del checklist.

Actividad 6. Prueba piloto y ajuste de instrumentos de medición

Se realiza una prueba piloto corta en ambos casos para validar que indicadores, checklist, plantillas y escenarios sean aplicables y consistentes. Se ejecuta una corrida sintética de baja carga para verificar captura estable de latencia, fallos y logs. Con base en los resultados se ajustan definiciones y criterios de registro para asegurar comparabilidad as-is/to-be. El piloto valida también el catálogo de error_type, el formato del Run Log/CR y la estabilidad del registro de timestamps por tramo.

- Técnicas y herramientas: ejecución controlada, bitácora de pruebas, hojas de cálculo, checklist, revisión de plantillas y escenarios, JMeter.
- Responsable: Elian Hernando Gil Sierra.
- Recursos: computador, n8n, hojas de cálculo, repositorio, almacenamiento para logs/resultados.
- Productos: instrumentos ajustados, criterios estabilizados, evidencia piloto, reporte breve de la prueba.

Actividad 7. Medición comparativa y recolección de evidencias (as-is vs to-be)

Se recolectan datos comparables para as-is y to-be en ambos casos bajo condiciones controladas. Se realizan ejecuciones repetidas con datos sintéticos y, cuando aplique, pruebas de baja y media carga. La información se consolida en un

dataset verificable, enlazado con decisiones documentadas, para interpretar el impacto con sustento.

- Técnicas y herramientas: hojas de cálculo, registro de ejecuciones, checklist, repositorio de evidencias (logs, outputs, exportaciones), JMeter.
- Responsable: Elian Hernando Gil Sierra.
- Recursos: computador, n8n, herramientas de registro, almacenamiento, datasets sintéticos.
- Productos: dataset comparativo por caso, evidencia de ejecuciones, resultados del checklist, reporte de pruebas sintéticas, aplicación de CR1–CR3 sobre as-is y to-be en ambos casos, registrando tiempos, nodos modificados, dependencias tocadas e intentos en CR Log. Consolidación de métricas finales en tablas comparativas almacenadas y contraste explícito frente a los criterios de éxito

Actividad 8. Evaluación arquitectónica por escenarios (ATAM) y análisis de trade-offs

Se aplica ATAM sobre as-is y to-be usando escenarios comunes y un árbol de utilidad para priorización. Se realizan walkthroughs por escenario y se registran riesgos, sensibilidades y trade-offs relacionados con atributos de calidad. Esto sustenta el análisis comparativo del micro-framework y ayuda a explicar resultados de la medición.

- Técnicas y herramientas: plantillas ATAM, sesiones de análisis, matriz de hallazgos, editor para reporte.
- Responsable: Elian Hernando Gil Sierra.
- Recursos: computador, documentación y diagramas as-is/to-be, plantillas ATAM, repositorio.
- Productos: ATAM aplicado y reporte comparativo por caso. Utility tree por caso, matriz de priorización (Impacto, Probabilidad, Prioridad), matriz de scoring escenario, registro de riesgos y trade-offs, tabla escenario/métrica.

Actividad 9. Diseño de arquitectura de referencia en AWS alineado con el micro-framework

Se elabora un diseño de referencia en AWS orientado a bajo costo y crecimiento progresivo, alineado con el micro-framework. Se documentan componentes,

puntos de escalado y controles básicos de seguridad, observabilidad y operación. Se incluyen escenarios de adopción con aproximaciones de costo basadas en supuestos explícitos.

- Técnicas y herramientas: diagramas AWS, documento técnico, matriz de escenarios, estimación de costos con calculadora AWS.
- Responsable: Elian Hernando Gil Sierra.
- Recursos: computador, diagramación, documentación AWS.
- Productos: diseño AWS documentado, diagramas, descripción, controles y escenarios de costo.

Actividad 10. Consolidación de guía de buenas prácticas y cierre del documento final

Se consolida una guía práctica de buenas prácticas para n8n a partir del micro-framework y la evidencia generada en los casos y en ATAM. Finalmente, se integra todo en el documento final del trabajo de grado, incluyendo anexos técnicos completos y trazables.

- Técnicas y herramientas: guía técnica, consolidación de anexos ADR, trazabilidad, checklist, ATAM, edición y normalización del documento final.
- Responsable: Elian Hernando Gil Sierra.
- Recursos: computador, Microsoft Word, repositorio, diagramación.
- Productos: guía de buenas prácticas v1.0 y documento final con anexos plantillas, matrices, diagramas, reportes y evidencias.